Changed Code Companion - v1.9

SIDM v19.2-F | Round 5 + n18 + n19 + n20 review response

Branch wip/v19.2-F-init

SIDM Round 5 + n18 + n19 + n20 review response — Consolidated Fixes Report (v1.11, 2026-10-26)

Scope: all confirmed Round 5 / local-review defects, with the code and data references that establish ground truth. Every number below was recomputed from source on 2026-10-25.

Branch: `wip/v19.2-F-init` Paper: `v0.3-prelim/docs/PAPER_V1_DRAFT.md` (2383 lines)

Supersedes v1.4 (2026-10-26). v1.4 was built on an invalid markup commit and is withdrawn. See §16 for the audit, the retraction, and the current state. The F1–F22 science content in §§0–15 is unchanged from v1.3 and was re-verified against the restored paper.

Supersedes v1.5 (2026-10-26) — see §17. v1.5 is withdrawn as the deliverable because the PDF builder used to render it had three silent content-loss defects (§17.2); the v1.5 markdown text is otherwise carried forward unchanged into this version. v1.5's science content is re-verified here and is unchanged.

Supersedes v1.6 (2026-10-26) — see §18. v1.7 adds the n20 review-response: an adjudication of all six of n20's action items and ten of its numeric claims, the resolution of the four section anchors n20 got wrong, an explicit "who owes it" column on the coverage-gap register, a "read this before trusting any 7/7 pass" declaration at the head of that register, one new tracked gap (CG-12, PDF-render integrity is not machine-checked), and a clarified verdict on n19 item 5. No scientific content changed and the paper is untouched by this version — n20 found no defect in the F23 fix and re-derived its numbers independently.

Supersedes v1.7 (2026-10-26) — see §19. v1.8 adds an independent verification of the n20 adjudication: a second agent re-derived all 14 of n20's substantive claims against on-disk ground truth with no knowledge of this report's conclusions. It confirmed 8 of this report's 10 adjudications, disputed 2 outright, produced 6 refinements, and — the reason this version exists — found 4 defects in this report's own work, two of which were cases where the response had absorbed n20's error rather than counting it independently. All 4 are fixed here: the namespace count (four → five), the diff's longest-line attribution (the paper diff's worst line is 3 582 characters, not the JSON diff's 1 035), n20's Fornax history error, and a fifth wrong section anchor (companion "§15", which does not exist). No scientific content changed and the paper is untouched by this version.

Supersedes v1.8 (2026-10-26) — see §19. v1.9 corrects the provenance claims in §19. The agent log shows the verifier was dispatched on Nemotron and fell back twice (`nemotron-3-ultra-550b-a55b:free → nemotron-3.5-lightning:free → deepseek-v4.1-flash`), and that the same effective model (`deepseek-v4.1-flash`) ran the n19 and third-pass audits — so §19's "a different model family" was wrong twice over: not deliberate, and not different. Three further corrections follow from the log: the call counts (27 API calls and 39 tool invocations, not "28 tool calls"), "edited no files" (it wrote three throwaway scripts in the scratch dir), and "a 16B-class dense model" (it is a 552B MoE with 8B/16B active). A dangling `§18.6` cross-reference — the exact failure mode §19 documents — was also removed. No scientific content changed and the paper is untouched by this version.

Supersedes v1.10 (2026-10-26) — see §21. v1.11 is the round in which the three items left to the user's decision in v1.10 were closed. CG-13 (the paper's 14 pointers to retired sections) is fixed: every one is now re-pointed at the section that holds the content, or dropped where the content had been deleted rather than moved (§10.4c,

§10.4d). CG-7 (the §10 L98 σ_eff band) is fixed by naming the omitted entry rather than widening the band. CG-11 (the permanently-red Table-walker) was a bug in the checker, not the paper, and is fixed — `run_round13_self_check.py` is now 8/8 green for the first time. Gate layer 8's stale-anchor baseline was lowered 14 → 0, so a single new pointer to a retired section fails the layer. Building that guarantee exposed a further silent-pass hole in layer 8 itself: its external-citation exemption was applied per line rather than per reference, so a stale anchor sharing a line with a ClawsGO citation was invisible (§21.3).

Supersedes v1.9 (2026-10-26) — see §20. v1.10 adds the three machine checks that close the loophole this document itself fell into. The v1.8 and v1.9 versions were corrected by hand after a late notification; nothing prevented the same class of error from shipping again. v1.10 adds three new gate layers — 8 (`audit_doc_anchors.py`: every `§N.M` in the report, the companion and the paper is resolved against the actual headings), 9 (`audit_model_provenance.py`: the reports' account of their own verification runs is checked against the delegation manifests, the agent log and the live transcripts), and 10 (`audit_hierarchy_chain.py`: the `σ/m(28) → ratio → g_N/g_χ` chain is recomputed from `constants.py` — this closes CG-9). The gate is now 10 layers, not 7. Building them also surfaced a new tracked gap (CG-13): the paper carries 14 references to sections that were retired (inherited from the 2026-10-05 copy-edit), which no existing layer noticed. No scientific content changed and the paper is untouched by this version — the paper's numbers, prose and conclusions are byte-identical to `e6b670c`, and the one paper-side finding (CG-13) is reported, not silently repaired.

Outcome (n19 review-response, v1.6): the n19 external review (Chinese-language, 181 paragraphs) raised 7 action items. 1 is acted on (a tracked coverage-gap list, §17.3); 3 are already addressed by the markup deferral and the existing guard work (§16.6, §15.1); 2 are documentation additions (a numbering-scope declaration, §17.4; a cross-platform note on the reviewer transport, §14.6); 1 is a misread of the PDF (§0 was never truncated — but checking it surfaced the three real builder defects, §17.2). Three of the reviewer's own supporting calculations were checked against source: 2 agree, and 1 has a wrong formula but the right conclusion (§17.5).

Out of scope for this cycle (stated so it travels with the document). Two things are deliberately not done here, both by explicit decision rather than oversight: (1) the literal-** markup pass stays deferred (per user direction, 2026-10-26) — §16.6 states the measured residual ("179 → 5-8 literals, 47 of 54 damaged paragraphs fully clean") and §16.7 states what is not claimed; (2) the coverage-gap register is a tracker, not a work plan — every row states its status, who owes it, and what would close it, and the OPEN rows are split between decisions owed by the user and tasks owed by the controller. A green gate does not close them.

Outcome (n20 review-response, v1.7): the n20 external review (Chinese-language, 178 paragraphs) marked all 11 prior-round items resolved or documented and raised 6 action items — nearly all about process (close or scope the tracked gaps) rather than artefact errors. 2 are already satisfied (the markup scope is stated; the CG-9 declaration existed, though not unmissably); 4 are acted on (an owner column on every OPEN row, the CG-9 declaration promoted to a standalone statement, the n19 item-5 verdict spelled out, and a plain-language index above the paper diff); 1 adds a new tracked gap (CG-12); and 1 asks for a closure timetable, which this document does not give — a date would be a commitment, not a fact, and no such commitment has been authorised. n20's own factual anchors are wrong in four places and its CG-10/CG-11 mapping is swapped; every one of its ten numeric claims was re-derived from source before any of it was accepted (§18.2).

Outcome (independent verification of the n20 adjudication, v1.8): a second agent re-derived all 14 of n20's substantive claims against on-disk ground truth with no knowledge of this document's conclusions. (It ran on the same effective model as the earlier audits — §19 corrects the original "different model family" claim, which was wrong.) It confirmed 8 of the 10 adjudications above, disputed 2 outright, and produced 6 refinements. More importantly it found 4 defects in this response — two of them cases where the response had absorbed n20's error rather than counting it independently (the namespace count, and the diff's longest-line figure). All 4 are fixed in v1.8: the namespace table defines five namespaces, not four; the paper diff's longest unwrapped line is 3 582 characters (not the JSON diff's 1 035); n20's Fornax history claim is wrong (the prior value was −0.47, not −0.534); and n20's "Companion §15" is a fifth wrong anchor (the companion tops out at §9). No scientific content changed and the paper is untouched by this version (§19).

Outcome (independent verification, v1.6): three independent audits of this deliverable each found something real that the author's own reading had missed — two documentation errors (the repair tool has three phases, not four; invariant 5 is advisory, not aborting) and one data error in the paper: F23, a stale `a_slope = 1.0` value (σ/m(28) = 65.49 at σ_1 = 1.0) that had been dismissed as a false positive in §14.4 and is now fixed in the paper (§17.7). The dismissal was self-refuting; the corrected value is 65.91, with the ratio and the hierarchy bound moving with it. New tracked gap CG-9 records why the then-7-layer gate never saw it. (CG-9 is now closed by gate layer 10 — §20.)

Outcome (Round 5, v1.0): 6 paper defects + 1 generator (root-cause) defect + 1 review-document defect class = 8 fixes applied, all verified. 2 additional pre-existing issues found and left alone (§10 L98 band; §9 Table-walker —

both confirmed pre-existing, not caused by these edits).

Outcome (n18 review-response, v1.1): the n18 external review raised 3 priority items against the v1.0 pair. All 3 are valid and all 3 are fixed (§13). Two supporting claims in the n18 review were corrected: (a) one of the three paper text errors was not introduced by Round 5 — it predates it by many commits; (b) the docstring mismatch is not a rounding-level difference — it is a 4.7% error in the quantity. The n18 review also produced 2 further structural improvements to the companion document, both applied (§13).

Finding-number namespaces (read this before any "F<n>" reference)

The labels F1–F22 are not one global sequence. They are five independent namespaces, one per review round, and several labels are reused across them. A bare F2 therefore does not identify a fix on its own — the section number is part of the identifier.

namespace	labels	defined in	refers to
Local review (Strata / qwen3.8-flash-next-iq3_xxs)	F2 (plus ERR L… / Q L…)	§8	the reviewer's own finding IDs in docs/LOCAL_REVIEW_R5.md
n18 review (Chinese-language)	F1–F7	§13.1, §13.3	the n18 review's 3 priority items + the fixes they prompted
Nemotron audit	F8–F9	§13.7	the Nemotron "Finding 1" mixed-anchor defect. (Note: F8/F9 are defined in §13.7 but also referenced in §14.2 and §14.3 — they are the same fixes, not a second set.)
Independent reviewer audit	F10–F16	§14.3	the σ_HL / stale-JSON defect class (§14.1)
Third reviewer pass	F17–F22	§15	the six further defects of §15

Rules for reading this document:

- §8's F2 is the local review's finding F2. It is not §13's F2 (the n18 cm2/g., punctuation fix). They are unrelated defects.
- F1–F7 in §13 mean the n18 set. F8–F9 in §13.7 mean the Nemotron set. F10–F16 in §14.3 mean the independent-reviewer set. F17–F22 in §15 mean the third-reviewer set.
- The fix IDs F1–F22 are contiguous and not ambiguous among themselves — the split above is by which review round produced them, and §14.3 continues to reference F8/F9/F14 rather than redefining them.
- A caveat on the §13.1 adjudication table: it ties fixes to n18 findings with an offset (n18 finding 3 → F4, 4 → F5, 5 → F6, 6 → F7), because F3 is an extra repair inside the same sentence as F1/F2. So Fn ≠ n18-finding-n for n ≥ 3.
- The phrase "F1–F22" (front matter, §16.3) means the union of all five namespaces — a shorthand for "every fix recorded in this report", not a single numbered sequence.
- When a fix is cited outside its defining section, cite it as §section.Flabel (e.g. "§14.3.F14"), which is unique.

This section was added in v1.6 in response to the n19 review's item 4, which correctly observed that a bare F1 is ambiguous across §§8/13/14/15.

0. TL;DR — what the reviewer got right, wrong, and what actually caused it

The local reviewer (Strata / qwen3.8-flash-next-iq3_xxs, 246.4 s, 14,292 prompt tokens, 1,816 completion tokens) produced 6 findings: 3 ERR, 3 Q. All 6 are actionable and all 6 are now fixed. But two of the reviewer's own supporting calculations were wrong, and the reviewer's headline diagnosis of the σ/m band was correct in conclusion while wrong in mechanism — the band is not a missing-assumption problem, it is a stale-value

problem.

The single root cause behind four of the six findings:

> Commit `79edeae` ("R88 full pre-submission: a_slope=1.93 propagation") changed `a_slope` from 1.0 to 1.93 in the numerical pipeline. It did not update the narrative strings in the generator `scripts/v192_dsph_gravothermal_sweep.py`, and the paper then quoted those stale strings verbatim.

So the paper said "σ/m ~ 0.5–2.5 cm2/g" and "|σ_HL| < 0.05 for 7 of 8 halos" while the committed JSON said 2.85–16.87 cm2/g and 0 of 8. The paper was faithfully quoting a stale generator, and the generator was faithfully re-emitting its own stale hardcoded text.

Fix strategy: fix the paper (6 edits) and fix the generator to compute its narrative from the results (root-cause fix), so the drift cannot recur. The generator's `dsph_halos` data is byte-identical before and after — only `verdict` and `limitations` changed.

1. FIX 1 — dSph σ/m band: `0.5–2.5` → `2.85–16.9 cm2/g`, with the f_H assumption stated

Where: paper L120 (excerpt `docs/R5_EXCERPT.md` tag L116)

Defect: the paper asserted `σ/m ~ 0.5–2.5 cm2/g` and `σ_eff ~ 0.03–0.10 cm2/g` at dSph scale, and offered the reconciliation "captured by three-term mixture with σ_HL typically small (|σ_HL| < 0.05 cm2/g for 7 of 8 halos)".

Ground truth (recomputed from `v0.3-prelim/data/results/v192_dsph_gravothermal_sweep.json` → `_derived_ranges`):

quantity	value	source		
framework σ/m at dSph scale	2.845 – 16.865 cm2/g	`sigma_m_paper_convention_cm2_per_g` over 8 halos		
published σ_eff	0.0318 – 0.0931 cm2/g	`sigma_eff_published_cm2_per_g`		
fitted σ_HL	−3.392 … −0.534 cm2/g	`sigma_HL_fit_cm2_per_g`		
halos with \	σ_HL\	< 0.05	0 of 8	`n_halos_abs_sigma_HL_below_0.05`

The reconciliation the paper missed — the two bands are not related by the two-component identity at the sweep's `f_H = 0.30`:

```
f_H^2 * sigma/m  =  0.09 * [2.845, 16.865]  =  [0.256, 1.518] cm^2/g
published band                                   [0.032, 0.093] cm^2/g
ratio above published  =  0.256/0.093 ... 1.518/0.032  =  2.8x ... 47.7x
```

i.e. the identity predicts 2.8–47.7× too much σ_eff. Bridging that gap requires a large NEGATIVE cross-channel term — not a "small positive" one. The paper's original claim that σ_HL was "typically small" was therefore wrong in both magnitude and sign.

Also fixed: the halo-mass assumption is now explicit — the three-term mixture is `σ_eff = f_H2σ_HH + 2f_Hf_Lσ_HL + f_L2σ_LL` with `f_H = 0.30` (T207, core-collapsed dSph; rounded from the canonical single-component `f_H = 0.297` of `constants.py`), `σ_LL = 0`, and σ_HL is a free fitted parameter, not derived from the framework.

Generator code — `scripts/v192_dsph_gravothermal_sweep.py`:

```
# BEFORE (hardcoded; stale since commit 79edeae)
{
    "interpretation": (
        "Per r20.docx Bug 1 fix: paper convention (Gaussian w=4.4) gives sigma/m ~0.5-2.5 cm^2/g "
        # ... two further string literals elided ...
        "sigma_HL required to reproduce sigma_eff is now near-zero or small positive -- consistent "
        "with a two-component model with f_H = 0.30."
    ),
}

# AFTER (computed from `results`; cannot drift again)
_rv = list(results.values())
_sm_lo  = min(r["sigma_m_paper_convention_cm2_per_g"] for r in _rv)
_sm_hi  = max(r["sigma_m_paper_convention_cm2_per_g"] for r in _rv)
_se_lo  = min(r["sigma_eff_published_cm2_per_g"]     for r in _rv)
_se_hi  = max(r["sigma_eff_published_cm2_per_g"]     for r in _rv)
_shl_lo = min(r["sigma_HL_fit_cm2_per_g"]            for r in _rv)
_shl_hi = max(r["sigma_HL_fit_cm2_per_g"]            for r in _rv)
_n_small = sum(1 for r in _rv if abs(r["sigma_HL_fit_cm2_per_g"]) < 0.05)
```

and the JSON now carries a machine-readable block:


```
{
  "_derived_ranges": {
    "note": "computed from dsph_halos; regenerate the file rather than hand-editing",
    "sigma_m_cm2_per_g": {"min": 2.8454570671911887, "max": 16.865226429437705},
    "sigma_eff_published_cm2_per_g": {"min": 0.0318, "max": 0.0931},
    "sigma_HL_cm2_per_g": {"min": -3.392310425355699, "max": -0.5340265143981119},
    "n_halos": 8,
    "n_halos_abs_sigma_HL_below_0.05": 0
  }
}
```


2. FIX 2 — "7 of 8 halos" → "0 of 8"; Fornax is the least discrepant, not an outlier

Where: paper L102 and L120 (excerpt tag L116)

Reviewer's question (L116): "'7 of 8 halos' appears again in a non-retracted context. L1012 retracts '7 of 8 channels', but L116 uses '7 of 8 halos' for |σ_HL|<0.05. Clarify whether this is a different quantity or a leftover retracted claim."

Answer: they are different quantities, so L116 was not a resurrection of the retracted L1012 claim — but the "7 of 8" count is now simply wrong.

- "7 of 8 channels" (L966, L993, L1065) = observational channels satisfied by the

retracted v18.29 / borrowed-`f_H` framing. Still present in the paper on purpose, each time marked as retracted, and the registry `PAPER_STANDING_NUMBERS.md` (lines 40, 189) correctly mandates retiring the "6–7 of 8" phrasing. Left untouched.

- "7 of 8 halos" (L116) = dSph halos in the σ_HL fit. A different object entirely.

But the halo count no longer holds: |σ_HL| ≥ 0.53 cm2/g for every halo, so 0 of 8 satisfy |σ_HL| < 0.05. And Fornax — described in the paper as "the outlier" at −0.473 — is in fact the least discrepant case: the JSON gives `sigma_HL_fit_cm2_per_g = -0.5340265143981119` for Fornax, and the `_derived_ranges` maximum (least negative) is exactly that value. The paper now says so.

3. FIX 3 — the `115–460×` factor range was computed from the RETRACTED A.12 table

Where: paper L939 and the threshold-rotation note at L129

Defect: §3.6 stated "If we instead use the velocity-independent limit (0.04 cm2/g), the violations are higher (115–460×)". Those numbers trace to the retracted A.12 σ/m table (σ/m = 18.4 and 4.6 cm2/g → 18.4/0.04 = 460, 4.6/0.04 = 115), not to the canonical table the paper now uses. They are not reproducible.

Corrected, recomputed from the canonical table (paper §3 table, values reproduced from `v0.3-prelim/code/channels_v03.py::sigma_m_at_v` + `constants.py`):

```
sigma/m(v) = sigma_m_at_v(SIGMA_0_CM2_PER_G, A_SLOPE, v) + 174.0*exp(-(v-29.4)^2/(2*4.4^2))

  v =  9  ->   5.428  /0.04 = 135.7x
  v = 40  ->   9.861  /0.04 = 246.5x
  => canonical range vs the 0.04 ceiling: 136-246x
```

The paper now says 136–246×, and states explicitly that the old 115–460× came from the retracted A.12 table.

Also fixed (L129): the threshold-rotation note said "the same σ/m(15) yields 3.6×, 10×, and 50× against the 0.8, 0.2, and 0.04 limits respectively." Recomputed with the canonical σ/m(15) = 2.845 cm2/g: 3.6×, 14.2×, 71.2×. The 10× and 50× were computed from a stale σ/m(15) ≈ 2.0. Corrected.

Verification of the paper's own σ/m table (all 8 rows, recomputed from code — all PASS):

v	code	paper
9	5.428	5.43
10	4.436	4.44
15	2.845	2.85
18	7.489	7.49
28	166.018	166.0
29.4	174.552	174.6
40	9.861	9.86

v	code	paper
100	0.052	0.052

4. FIX 4 — 27–49× is the v=9 / v=40 ENDPOINT PAIR, not a min–max over the window

Where: paper L40 (abstract-adjacent main result), and the two summary restatements at L2136 and L2193

Defect: "27–49× at v = 9–40 km/s" reads as a min–max over that velocity window, but the window's actual minimum is 14× at v = 15. The 27× and 49× are the two endpoints.

Full canonical row set vs the 0.2 cm2/g ceiling:

v	σ/m (cm2/g)	vs 0.2	note
9	5.428	27.1×	endpoint (Sculptor)
10	4.436	22.2×	
15	2.845	14.2×	window minimum
18	7.489	37.4×	
28	166.018	830×	excluded (resonance)
40	9.861	49.3×	endpoint (Cluster LMC)

The paper now labels 27–49× as the endpoint pair and states the 14× window minimum. All three restatement sites (L40, L2136, L2193) carry the same label, so the paper no longer uses the range two different ways.

Reviewer's arithmetic, corrected: the review proposed the lower end should be "~22×" (from 4.39/0.2 = 21.95). The true window minimum is 14× (2.845/0.2 = 14.2).

5. FIX 5 — the σ_1 hierarchy-constraint contradiction (review ERR-3), resolved with computed bounds

Where: paper L151 (excerpt tag L145)

Defect (confirmed, and sharper than the review stated): the paper said "Both new values are less restrictive than the σ_1 = 4.4 headline" and then "under σ_1 ≲ 3.0, the constraint is tighter". But σ_1 = 1.0 is inside "σ_1 ≲ 3.0", so one sentence assigned opposite properties to a single regime. The review's hedge ("unless it refers to a different σ_1 regime not shown") is the charitable reading; the plain reading is a contradiction.

Computed from the paper's own hierarchy formula (`g_N/g_χ < 1/√(σ/m·m_χ/σ_SI)`, `m_χ = 1.78e-24 g`, `σ_SI = 9.4e-48 cm2`, anchor v = 28):

σ_1	σ/m(28)	g_N/g_χ
4.4	166.018	1.784e-13
3.0	156.655	1.836e-13
1.0	65.911	2.831e-13

Narrowing σ_1 lowers σ/m at v = 28 and therefore loosens the allowed g_N/g_χ — a larger permitted g_N/g_χ is a deeper dark-sector hierarchy, not a tighter constraint on it. The paper now states the three computed bounds and the direction of the shift.

Anchor bookkeeping (found during verification): the paper's headline `1.74e-13` is quoted at the resonance-peak anchor (σ_peak = 174 cm2/g at v = 29.4), whereas `2.83e-13` is quoted at the v = 28 anchor — so the printed pair compares two different anchors. Like-for-like at v = 28 the baseline is 1.78e-13. The direction is unchanged either way (D-5 looser), so the conclusion stands; the paper now states this explicitly rather than leaving the mismatch implicit.

Source of the numbers: `scripts/apply_round2_paper_fixes.py` lines 31–45 define the hierarchy formula and carry their own asserts:

```
def hier(sigma_m):
    ratio = sigma_m * m_chi_g / sig_SI
    return ratio, 1.0 / math.sqrt(ratio)

r28can, g28can = hier(174.0)  # headline anchor: sigma_peak = 174
assert abs(g28can / 1.742e-13 - 1) < 1e-2, "headline g must match L2107 (1.7e-13)"
assert g28d5 > g28can, "D-5 v=28 bound must be looser than the headline"
```

6. FIX 6 — the `1/σ_peak` interpolation assumption, now stated and validated

Where: paper L195 (excerpt tag L189)

Defect (review Q, correct): "causality requires σ_peak ≤ 47.67 (interpolated from ratio = 2.86 at σ_peak = 50)" silently assumes `ratio ∝ 1/σ_peak`.

The assumption holds on the grid. With `ratio = 143/σ_peak` (143 = 2.86 × 50):

σ_peak	predicted	table	diff
50	2.860	2.86	0.0%
150	0.953	0.96	0.7%
174	0.822	0.83	1.0%
200	0.715	0.72	0.7%
250	0.572	0.58	1.4%

and `143/3.0 = 47.67` reproduces the cap exactly. The paper now states the assumption and the grid check.

7. FIX 7 (root cause) — the generator now computes its own narrative

File: `scripts/v192_dsph_gravothermal_sweep.py`

Five edits, no line moves, `py_compile` clean:

1. Docstring conclusion rewritten (it asserted "the 100x inconsistency largely disappears" and "σ_HL ... is now near-zero or small positive").
2. Console print was hardcoded to `sigma/m ~0.6 cm^2/g` → now `{_sm_lo:.2f}-{_sm_hi:.2f}`.
3. `verdict.interpretation` → computed f-string (§1 above).
4. `_derived_ranges` block added to the JSON output.
5. `limitations` gained a caveat naming the halo-mass assumption and the superseded band.

Data integrity: `dsph_halos` and `framework_parameters` are byte-identical before and after; only `verdict` and `limitations` changed. Verified by direct JSON comparison against the pre-edit copy.

8. FIX 8 — the review document's own errors

File: `docs/LOCAL_REVIEW_R5.md` — all 6 findings annotated inline with Audit note (2026-10-25), plus an audit header.

#	Reviewer finding	Verdict
F2 (local-review label)	ceiling labels on L36	OK — the stray trailing `[Q]` removed; the "9.85" arithmetic flagged
ERR L36	27–49× unsupported by excerpt	VALID, proposed lower end superseded (14×, not 22×)
ERR L117	0.5–2.5 conflicts with 5.43/4.39	VALID and worse than stated — stale band, not a missing assumption; line ref off by one
Q L117	"7 of 8 halos" vs retracted "7 of 8 channels"	ANSWERED — different quantities; and the count is 0 of 8
ERR L145	hierarchy contradiction	CONFIRMED, sharper — σ_1 = 1.0 is inside "σ_1 ≲ 3.0"
Q L189	1/σ_peak assumption unstated	CORRECT, assumption validated to ≤1.4%
Q L2210	archival ranges	ANSWERED — both reproducible from the canonical table; the stale one was 115–460×

Namespace note: the labels in this table are the local review's own finding IDs (from `docs/LOCAL_REVIEW_R5.md`, where they appear as `[OK] F2`, `[ERR] …`, `[Q] …`). They are not the report's fix IDs: the fix-ID `F2` in §13 is a different

defect (the `"cm2/g.,"` doubled punctuation). See the "Finding-number namespaces" table in the front matter.

Two of the reviewer's own supporting calculations were wrong:

- "9.85/0.2 = 49.25×" — there is no 9.85 cm2/g anywhere in the paper. L143's `9.85e+21` is a dimensionless ratio; the v=40 σ/m is 9.86. The correct division is `9.86/0.2 = 49.3×`.
- "~22× lower end" — 22.2× is the v=10 value; the window minimum is 14.2× at v=15.

Line references: the review cited "L117" for text that is at excerpt L116 / paper L120 — off by one.

9. Verification summary

Recomputed from source, 2026-10-25 (all PASS):

- 8/8 paper §3 σ/m table rows reproduced from `channels_v03.sigma_m_at_v` + `constants.py`
- factor ranges: 136–246× (vs 0.04), 27–49× endpoints (vs 0.2), 14× window min, 3.6–23.7× (vs 0.8)
- σ/m(15) → 3.6× / 14.2× / 71.2× against 0.8 / 0.2 / 0.04
- 1/σ_peak assumption: 5/5 grid points within 1.4%; cap 143/3.0 = 47.67
- hierarchy: 1.784e-13 / 1.836e-13 / 2.840e-13, monotonically loosening
- JSON `_derived_ranges` ↔ paper text: exact agreement on all 6 values
- stale strings eradicated: `0.5–2.5` (1 remaining = the explicit correction note), `115–460×` (1 remaining = the explicit correction note), `7 of 8 halos` (0), `−0.473` (0)

Project audit gates (`mcp__sidm_test__sidm_run_all_audits`):

- `drift_guard` — PASS, 85/85 checks, no drift
- `claims_table` — PASS, 29 pass / 0 fail ("All numbers verified against source JSON. Zero drift.")
- `self_check` — 7/8 pass. The one failure is Table-walker at line 483, and it is pre-existing: `git stash` of the paper reproduced the identical failure on the committed version. Not caused by these edits. (The `self_check` also has a pre-existing cp1252 decode bug in `scripts/run_round13_self_check.py` — it must be run with `PYTHONIOENCODING=utf-8 PYTHONUTF8=1`; confirmed pre-existing by testing the committed paper.)
- `pytest test_paper_claims.py` — 12 passed

10. The §10 L98 band (CG-7) — fixed, with the band retained

Paper L98 (before): "The framework's published σ_eff values (`sigma_m_phase44.json`) are 0.03–0.10 cm2/g at dSph/UFD velocities."

Recomputed from `v0.3-prelim/data/results/sigma_m_phase44.json`, the five dSph/UFD entries are `v3.0_extreme_UFD = 0.15488440893348154`, `v5.0_UFD = 0.09305390842195108`, `v7.0_edge_UFD = 0.06659099956169537`, `v10.0_UFD = 0.04680728549612335`, `v15.0_classical_dSph = 0.031775504769417776` → the full span is 0.0318–0.1549, i.e. the quoted `0.03–0.10` covered v = 5–15 and omitted the extreme UFD at v = 3 km/s (0.155).

Fix applied (the option recommended in v1.10): the band is retained and the omitted entry is named. The paper now reads:

> ...are 0.03–0.10 cm2/g at dSph/UFD velocities — the quoted band covers v = 5–15 km/s; the full phase44 dSph/UFD entry set spans 0.032–0.155 cm2/g, the top being the extreme UFD at v = 3 km/s.

Why retain the band rather than widen it to 0.03–0.16: `0.03–0.10` is the conventional dSph-velocity range and is correct as stated for v = 5–15; the v = 3 km/s entry is an extreme UFD, a different population. Widening the printed band to include it would silently change what the sentence is a statement about. Naming the entry keeps both facts and leaves the convention intact.

Verification: the five values above are read from the committed JSON at report-build time, not copied; the paper's text is checked against them by gate layer 5's source registry.

11. Files changed

file	change
`v0.3-prelim/docs/PAPER_V1_DRAFT.md`	6 fixes (§1–§6) + 2 consistency edits (L2136, L2193)

file	change
scripts/v192_dsph_gravothermal_sweep.py	5 edits — narrative now computed (§7)
v0.3-prelim/data/results/v192_dsph_gravothermal_sweep.json	regenerated: verdict + limitations + _derived_ranges; dsph_halos unchanged
docs/LOCAL_REVIEW_R5.md	6 findings annotated + audit header (§8)
docs/dual_review_5loop/SIDM_ROUND5_CONSOLIDATED_FIXES_v1.0_2026-10-25.md	this report

12. Code / data reference index

- v0.3-prelim/code/channels_v03.py → sigma_m_at_v(S0, A_SLOPE, v) — canonical σ/m(v)
- constants.py → SIGMA_0_CM2_PER_G, A_SLOPE (= 1.93) — canonical constants
- v0.3-prelim/code/phase44_two_component.py — three-term mixture
- v0.3-prelim/code/T206_f_H_fit.py — f_H fit
- scripts/v192_dsph_gravothermal_sweep.py — dSph sweep generator (the fixed one)
- v0.3-prelim/data/results/v192_dsph_gravothermal_sweep.json — per-halo ground truth
- v0.3-prelim/data/results/sigma_m_phase44.json — published σ_eff velocity grid (§10)
- scripts/apply_round2_paper_fixes.py — hierarchy formula hier(), lines 31–45
- scripts/apply_round5_paper_fixes.py — Round 5 paper-fix implementation
- scripts/r5_local_review.py — local reviewer driver (Strata primary, 35B standby)
- scripts/local_model.py — local model registry (FAST :11437, DEEP Strata :8080)
- docs/LOCAL_REVIEW_R5.md — the audited local review
- docs/R5_EXCERPT.md — the excerpt the review saw (its own line tags)
- docs/dual_review_5loop/FIXING_ROUNDS_REPORT.md — Round 5 adjudication
- docs/dual_review_5loop/GROUND_TRUTH.json, ADJUDICATION_EXTRA.json — adjudicated ground truth
- v0.3-prelim/docs/PAPER_STANDING_NUMBERS.md — claim registry (29/29 verified)

13. n18.docx review response (v1.1) — adjudication and fixes

Source: n18.docx (external review of the v1.0 report + companion, in Chinese).

Important note on the source document: the n18 .docx is itself truncated. The exact recoverable/unrecoverable boundary (stated precisely in v1.6, per the n19 review):

part of the n18 review	status				
severity table, row 7 — 7 diff 单行长段落可读性差 低 考虑分行 diff 或高亮	recoverable (it is the table's final row)				
severity table, findings 1–6	NOT recoverable — the rows are absent from the package; the file holds only 23 text nodes / 1 527 characters (after XML-entity unescaping). The file's last extractable text is `	σ_HL	10}'7 diff 单行长段落可读性差 低 考虑分行 diff 或高亮 — the dangling	σ_HL	10}'` is itself the truncation artefact

part of the n18 review	status				
closing 总体判断 section — the 3 priority items	recoverable and complete — these are the actionable set addressed here				
the two supporting claims later corrected as C1 and C2	recoverable — quoted inline in the text below				

Consequence for a reader: n18's severity grading for findings 1–6 cannot be checked, because the grading did not survive in the file. The substance of those findings does survive, and all three priority items are addressed below. By contrast the n19 review (see §17) is not truncated — 181 paragraphs, 6 928 characters (convention: 6 748 decoded characters plus 180 newline separators; raw `<w:t>` count 7 021; WPS `CharactersWithSpaces` 6 748), all pages read.

13.1 Adjudication table

#	n18 finding	Controller verdict	Provenance (independently re-derived)	Action		
1	Paper: `"halos is"` word order	VALID	Introduced by Round 5 — `git log -S"halos is"` → `7cc8d8a`; absent from `7cc8d8a^` (which read "For 7 of 8 halos, \	σ_HL\	< 0.05 cm2/g")	F1 fixed
2	Paper: `"cm2/g.,"` doubled punctuation	VALID	Introduced by Round 5 — `git log -S"cm2/g.,"` → `7cc8d8a` only; count in the paper = 1	F2 fixed		
3	Paper: `"and 4 baseline"` stray chars	VALID defect, provenance CORRECTED	Pre-existing since `799f4ef` ("R88 paper cleanup"), NOT Round 5. `git log -S"changes and 4 baseline headline"` → `799f4ef`; the same string is present in `7cc8d8a^`	F4 fixed		

#	n18 finding	Controller verdict	Provenance (independently re-derived)	Action		
4	Generator docstring `"1.589"` vs computed `"1.518"`	VALID, severity CORRECTED	Introduced by Round 5 (`git log -S"0.257-1.589"` → `7cc8d8a`). Not "rounding-level": 1.589 implies σ/m = 17.656 cm2/g vs the canonical 16.865 — a 4.7% error. The lower end 0.257 implies 2.856 vs canonical 2.845 — also wrong	F5 fixed + regression test		
5	Companion embeds 241 lines of unchanged JSON	VALID	Verified: the embedded block is byte-identical to the live JSON (`d == live` True, all 8 top-level keys equal, zero differing values) — it carries no information the live file lacks	F6 fixed		
6	Diff single-line long paragraphs (low)	VALID	Max line length in the v1.0 diff fence = 3 581 characters	F7 fixed		

13.2 Two corrections to the n18 review

C1 — "these are new errors introduced by the fix (not present in the original paper)" is wrong for finding 3. The `"and 4 baseline"` defect was introduced by commit `799f4ef` and is present in `7cc8d8a^` unchanged; Round 5 carried it through untouched. The defect is real and is fixed, but attributing it to Round 5 would send the next reviewer after the wrong commit.

Root cause (verified against `799f4ef^`): that commit replaced the phrase `"the ratio derivation changes and g_N/g_χ < 10-23. The σ_1 = "` with `"the ratio derivation changes and "`, deleting the trailing `"The σ_1 = "` of the following sentence and leaving `"4 baseline headline value"`. The restoration used is `"the σ_1 = 4.4 baseline headline value"` — the paper's own σ1 = 4.4 baseline, consistent with the computed like-for-like bounds in the same paragraph (1.78e-13 / 1.84e-13 / 2.83e-13).

C2 — the docstring gap is not a rounding difference. n18 described `"1.589"` vs `"1.518"` as 小数舍入级别 (decimal-rounding-level). It is not. `0.09 × 16.865226 = 1.517870` → 1.518; the quoted 1.589 implies `0.09 × 17.656`, i.e. a σ/m maximum 4.7% above the canonical value. Recomputation from two independent sources agrees on 0.256–1.518: `channels_v03.sigma_m_at_v` + `constants.py` (σ/m(5) = 16.865226 → 0.09× = 1.517870) and the JSON's `_derived_ranges.sigma_m_cm2_per_g` (2.845457..16.865226 → 0.256091..1.517870).

13.3 The fixes

Fix	File	Change				
F1	`v0.3-prelim/docs/PAPER_V1_DRAFT.md` L102	`"For 0 of 8 halos is \	σ_HL \	< 0.05 cm2/g" → "For 0 of 8 halos, \	σ_HL \	< 0.05 cm2/g"` (drop the dangling "is", restore the comma)
F2	same, L102	`"σ_HL = −0.534 cm2/g., indicating"` → `"σ_HL = −0.534 cm2/g, indicating"` (drop the stray period)				
F3	same, L102	`"indicating an**internal inconsistency"` → `"indicating an **internal inconsistency"` (missing space; pre-existing, but inside the sentence F1/F2 repair)				
F4	same, L151	`"changes and 4 baseline headline value"` → `"changes and the σ_1 = 4.4 baseline headline value"` (restore the deleted subject)				
F5	`scripts/v192_dsph_gravothermal_sweep.py` L24	docstring `"0.257-1.589 cm^2/g"` → `"0.256-1.518 cm^2/g"`				

Fix	File	Change				
F5b	`v0.3-prelim/tests/test_n18_generator_docstring_consistency.py` (new)	4 regression tests pinning the docstring's quoted numbers to the generator's own output JSON				
F6	companion §3	241-line full-JSON embed → a 39-line real `git diff` (pre-Round-5 → current)				
F7	companion §2	diff fence hard-wrapped at 108 columns (was up to 3 581 chars/line)				

Fix script: `scripts/apply_n18_review_fixes.py` — idempotent (each replacement asserts exactly one occurrence of the old string and refuses to edit otherwise), and its module docstring records each finding, its independently-re-derived provenance, and the rejected items.

13.4 Why F5 is a generator fix and not a docstring edit

Per the stale-narrative-string-audit discipline: a docstring that restates a computed quantity is a second copy of the defect and will drift again. Round 5 made the JSON's `interpretation` and `limitations` fields computed but left the docstring hand-written — so the fix set had a remaining hole. F5 therefore also adds a regression test that recomputes `F_H_CC**2 × _derived_ranges.sigma_m_cm2_per_g` from the generator's own output and asserts the docstring quotes the same numbers.

The test is mutation-verified: reverting the docstring to `0.257-1.589` turns 2 of the 4 tests red (the consistency test and the explicit retracted-value guard); restoring it turns them green again.

The test also reads `F_H_CC` from the generator's own constant rather than hard-coding 0.30, so it tracks the generator if f_H ever changes.

13.5 What was NOT actioned (and why)

- Bold-marker spacing (e.g. `an**internal`, `is**NOT falsified**`; 270 occurrences of a `**`

with word characters on both sides). These render correctly — verified with `markdown_it` (the CommonMark reference implementation): `"an**internal inconsistency**"` renders as `<p>indicating an<strong>internal inconsistency</strong> end</p>`, the emphasis is applied and no literal `**` survives. This is the paper's pre-existing house style across ~270 sites, it was not flagged by n18, and a wholesale restyle is a large mechanical rewrite of a 2 383-line paper. Only the single instance inside the sentence being repaired (F3) was fixed.

- The paper's `"does not.."` (line 974) and `"curves .**"` (line 120) — both pre-existing

(`650ff13` and `799f4ef`), outside the reviewed diff, cosmetic. Recorded for a future sweep rather than mixed into this round.

13.6 Verification of the n18 fix set

check	result
n18 regression test (4 tests)	4/4 PASS; mutation-verified (2 turn red on the retracted value)
paper: new strings present, old strings gone	3/3 present, 4/4 old strings count = 0

check	result
rendered check (`markdown_it`, CommonMark)	all 3 repaired sentences render correctly; no literal ** survives
companion §1 listing vs live generator	byte-identical (288/288 lines)
companion §2 diff fence	lossless — unwrapping reconstructs the true `git diff` exactly (83/83 lines)
companion §3 diff fence	lossless — unwrapping reconstructs the true `git diff` exactly (39/39 lines)
fix script idempotency	re-run applies 0 changes (all 5 report "already applied")

13.7 Additional fixes from Nemotron reviewer audit (Finding 1)

Nemotron Finding 1 (partly-wrong → actionable): The paper's §3 channel table (L115) paired the canonical σ/m = 2.85 cm2/g with the §2.6a sweep's σ_HL = -0.47 — a mixed-anchor row. Recomputing from that pair gives σ_eff = 0.0538 (not the published 0.032). The correct canonical σ_HL for σ/m = 2.8455 is −0.534. The sweep's -0.47 is self-consistent at its own σ/m = 2.5597.

Fix	File	Change
F8	`v0.3-prelim/docs/PAPER_V1_DRAFT.md` L115	`"σ_HL = -0.47"` → `"σ_HL = −0.534, canonical sweep"` (match the row's canonical σ/m anchor)
F9	same L221	`"σ_HL_required = −0.47 cm2/g"` → `"σ_HL_required = −0.47 cm2/g, §2.6a σ_peak sweep"` (explicit disambiguation)

Scope note: L190 (the §2.6a sweep table row for σ_peak=174) and L194 (sweep discussion) are already in the correct sweep context and need no change. The pre-existing L120 grep hit was a false positive.

Verification:

```
# L115 row: canonical sigma/m=2.8455, published sigma_eff=0.0318 -> required sigma_HL = -0.5340 ✓
# Sweep: sigma/m=2.5597, sigma_HL=-0.4723 -> sigma_eff=0.03201 ✓
```

14. Independent reviewer audit — corrections and additional fixes (v1.2)

Two independent reviewers audited the v1.1 fix set. One of them found a material defect that the controller had missed, and the controller's own earlier verification of a second class of issue turned out to be wrong. Both are recorded here in full.

Reviewer A — Nemotron 550B (`nvidia/nemotron-3-ultra-550b-a55b:free` via OpenRouter, delegated with no access to the controller's reasoning). Output archived at `docs/NEMOTRON_N18_FIX_AUDIT_2026-10-25.md`. Verdict: all 6 claims VERIFIED; 3 additional findings raised.

Reviewer B — `opencode/deepseek-v4.1-flash-free` (opencode CLI, run with the excerpt files attached rather than pasted into argv — see 14.6). Output archived at `docs/OPENCODE_N18_FIX_AUDIT_2026-10-25.md`. Verdict: 4 of 6 claims VERIFIED, 2 PARTLY-WRONG; plus questions D1–D3 and additional findings. Reviewer B is the reviewer that found the material defect in 14.1.

14.1 CONFIRMED MATERIAL DEFECT — the §2.6a Fornax σ_HL column was stale (Reviewer B, D1)

This is the most serious finding of the whole round and it was missed by the controller.

The §2.6a findings table in `PAPER_V1_DRAFT.md` has a "Fornax σ_HL req" column. Its values came from `v192_a_phase44_sigma_peak_sensitivity.json`, which was stale: the checked-in copy predated the R88(26) change of the resonance anchor from `v = 28` to the canonical `v = 29.4`. Evidence — the checked-in JSON's own `parameterization` field read `baseline + sigma_peak*exp(-(v-28)^2/(2*4.4^2))` while the generator's source reads `V1_V_TARGET = v192_dsph.V1_V_TARGET # 29.4 km/s` with the comment `# R88(26) fix: 28 -> 29.4 canonical`.

Effect on the Fornax σ_HL column (σ_peak = 174):

	σ/m(15)	required σ_HL
stale checked-in JSON (superseded resonance anchor v = 28)	2.5597	−0.4723
regenerated (canonical v = 29.4)	2.8455	−0.5336

That is a 13 % shift, not rounding. The full column moved from `−0.08, −0.13, −0.20, −0.27, −0.34, −0.41, −0.47, −0.54, −0.68` to `−0.39, −0.41, −0.43, −0.46, −0.48, −0.51, −0.53, −0.56, −0.61`.

A narrative claim was consequently false. The paper's §2.6a finding 3 read "At σ_peak ∈ {30, 50}, σ_HL_required is marginal". Under the canonical generator the marginal set is empty — the generator prints `marginal (within 0.15 of zero) for sigma_peak in []`; and every one of the nine swept values is substantive. The old values only looked marginal because the stale anchor made them smaller in magnitude.

How the drift survived: the paper's σ/m columns were regenerated (R88(31)) and match the current generator to ~0.03 %, but the σ_HL column was carried over from the pre-R88(26) run — so a single table row mixed two parameterizations. The generator's own printed narrative also carried hardcoded values (`-0.08 … -0.68`), i.e. the same stale-narrative-string class as the Round-5 root cause.

14.2 Corrections to the controller's own v1.1 claims

(a) §13.5 — "the bold markers render correctly" is WRONG. That claim was verified against a synthetic string. Re-tested against the paper's real text with the CommonMark reference implementation (`markdown_it`):

- `calibrated**independently**` renders as `calibrated<strong>independently</strong>` — the

emphasis applies but the words are glued ("calibratedindependently").

- `constraint**depends on the anchor.` renders with the literal `**` surviving.
- `the paper.**Breit-Wigner peaks:**Each` renders with the literal `**` surviving.

Measured on the live paper: 70 lines whose rendered output still contains a literal `**`, and 618 glued-word groups. 22 of the 70 broken lines contain backtick code spans, and several carry 13–38 `**` markers. Publicly withdrawn as stated in §13.5.

(b) §13.7 — the "different parameterizations" framing is WRONG. The controller adjudicated −0.47 as a legitimate alternate parameterization from the §2.6a sweep. It is not: it was an artifact of the stale JSON (14.1). F8/F9 were therefore directionally right (the correct value is −0.534/−0.53) but their reasoning and their "sweep vs canonical" contrast labels were wrong. Both labels are now removed (F14) and the values unified.

(c) Companion counts. The companion quotes "241 lines, 9,895 bytes" for the dSph JSON and "39 lines changed". Measured: the live file is 241 lines, 10,135 bytes, and the true `git diff` is 41 lines total (4 header + 37 body; +19/−1). The byte figure is stale (the Round-5 narrative regeneration enlarged the file) and neither 39 nor the byte figure matches the current state. Corrected in the companion.

14.3 Fixes applied (F10–F15)

Fix	File	Change
F10	`v0.3-prelim/data/results/v192_a_phase44_sigma_peak_sensitivity.json`	regenerated from the current generator (canonical v = 29.4 anchor)
F11	`scripts/v192_a_phase44_sigma_peak_sensitivity.py`	the hardcoded Fornax narrative (`-0.08 … -0.68`) → computed from `results`; emits `-(0.39) … -(0.61)`
F12	`PAPER_V1_DRAFT.md` §2.6a table (9 rows)	Fornax σ_HL column → canonical values; marginal/substantive labels → all substantive

Fix	File	Change
F13	same, finding 3	"marginal at σ_peak ∈ {30, 50}" → "substantive at every σ_peak (30–250); no σ_peak leaves the Fornax σ_HL marginal"
F14	same, L120	`σ_HL = -0.47` → `−0.534 cm2/g`
F15	same, L221 + L115	`−0.47 … ,` §2.6a `σ_peak sweep` → `−0.53 cm2/g at σ_peak = 174`; drop the F8/F9 contrast labels
F16	`v0.3-prelim/tests/test_n18_sigma_hl_column_matches_json.py` (new)	4 regression tests pinning the table's σ_HL column and its labels to the JSON

The missing guard, now added. Nothing in the suite connected the paper's §2.6a σ_HL column to the JSON it is read from, which is why a 13 % error sat in the paper unnoticed.

Direct proof that the gate was blind to this class. The full 7-layer pre-tag gate was run twice — once at 22:03:20 (before the fix) and once after `1c07d65` (committed 23:33:19). Both runs report `ALL GATES PASSED — safe to tag a new release`. The 22:03 run therefore certified a paper whose §2.6a σ_HL column was 13 % wrong and whose finding text asserted a false marginality. The reason is visible in the gate's own output: `audit_claims.py` scans 1,034 numeric lines but states "only the headline Phase claims are tracked", and its source contains zero references to `sigma_HL` or §2.6a. So `7/7 PASS` never covered this table, and the new test is not redundant coverage — it is the first check that reads this column at all.

Generalisation: a green pre-tag gate is evidence about the claims the gate enumerates, not about the document. When a numeric defect is found in a document a gate certifies, the finding to carry forward is the gate gap, not just the fixed cell — and the gate's summary line (`ALL GATES PASSED`) must not be quoted as if it covered the defect. F16 reads both sides at test time (so it is not a hard-coded literal) and is mutation-verified on three failure modes: reverting one table cell to `−0.47` fails 2 of 4 tests; flipping one label fails 1; perturbing the JSON value at σ_peak = 174 fails 1. Restoring both files byte-identically returns 4/4 green.

14.4 Reviewer findings adjudicated as FALSE POSITIVES

- ~~Reviewer A #2 — "σ/m(28) = 65.49 is ~0.6 % low vs the generator formula". Refuted.~~

WITHDRAWN — Reviewer A was right and this dismissal was wrong (see §17.7, finding F23). The original reasoning was self-defeating: it computed the canonical value `0.052·(28/100)^-1.93 + 174·exp(-(28-29.4)2/2) = 0.6067 + 65.3041 = 65.9109` and then asserted, in the same sentence, that 65.49 was the correct value "once the baseline term is evaluated at the same anchor". Those cannot both be true. 65.49 = 0.052·(28/100)^-1.0 + 65.3041 = 65.4899 — i.e. it embeds the retracted `a_slope = 1.0` baseline term, not the canonical `a_slope = 1.93` one. The paper's own "like-for-like v = 28" list had two canonical values (166.0, 156.7) and one stale one (65.49) in the same clause, which is the tell. F23 fixes the paper. The other three items in this section stand.

- Reviewer A #3 / Reviewer B claim 6 — "the companion shows literal `\n` in the JSON diff".

Refuted as a fidelity issue. The true `git diff` shows the same `\n` because the JSON string contains an escaped newline; any faithful diff renders it that way.

- Reviewer B D1 arithmetic. The reviewer's own inversion is correct

(`(0.0318 − 0.09·2.845457)/0.42 = −0.53403`), which is what confirmed the finding.

- Reviewer B claim 3 — "7.5×10-12 is inconsistent with the 1.78e-13 v = 28 bounds".

Not a defect. They are different regimes: 7.5×10-12 is the σ_1 = 4.4 baseline headline (anchored to σ_DM-DM ~ 1 cm2/g), while 1.78e-13/1.84e-13/2.83e-13 are the σ_1 = 1.0 re-anchoring at v = 28. The sentence states the re-anchoring; comparing across the two regimes produces the reviewer's factor of 42. Legitimate readability note only.

14.5 Findings NOT actioned, with reason

- The 70-line literal-** markup defect (14.2a). Real, but not mechanically fixable.

A minimal "insert a space around the delimiter" rule was tested against all 70 lines: it fixed 0 of 70 and changed content beyond whitespace on 69 of 70, inserting spurious spaces inside backtick code spans (e.g. inside `σ0(v) = σ0 · (1 km/s / v)^a`). The affected lines are structurally complex — 22 contain code spans and several carry 13–38 markers. Correcting this needs a context-aware editorial pass, not a regex, and must not be bundled into a numerical-fix commit. Recommended next step: a dedicated pass with a per-line renderer assertion (`no literal ** in render` AND `visible text unchanged`), reviewed as its own diff.

- Residual ~0.03 % σ/m implementation difference. The paper's σ/m columns agree with the

regenerated JSON only to the second decimal in a few cells (e.g. paper `161.70` vs JSON `161.658`; `93.14` vs `93.1173`). Hand-evaluating the documented canonical formula gives 161.70, so the paper matches the formula and the JSON's `sigma_m_at_v` helper differs marginally. Sub-display-precision in most cells and not material, but the two implementations should be reconciled in a dedicated pass.

- `does not.` . (L974) and `curves .**` (L120). Pre-existing, outside the reviewed diff.

Note that L120 is inside the §2.5 region that F14 touched, so a future sweep should take it then.

14.6 Reviewer-transport fix (why Reviewer B initially produced nothing)

`scripts/dual_review.py` passed the prompt as a command-line argument to `opencode run`. A 23 KB prompt exceeds the Windows command-line limit: all five reviewer models returned `EXIT_1 The command line is too long` and no review was written. The failure also misreports itself — five models dying in 0.1 s each reads as five dead models, not as one oversized argument.

The driver is now fixed (`build_opencode_argv`, `MAX_ARGV_PROMPT = 6000`): a prompt longer than the threshold is written to a temp file and attached with `-f`, with a short instruction stub as the message. Verified end-to-end on the exact prompt that failed — the 23 KB prompt now returns `OK` through the unmodified CLI path, argv reduced from 22,387 to 536 chars. Two traps are recorded in the helper's docstring:

```
opencode run --model <id> --format json "$(cat short_prompt.txt)" \
  -f excerpt_paper.txt -f excerpt_sweep.txt > review.jsonl
```

- `-f` is an array option — it swallows a later positional message, so the message must

come first. (Also note `opencode run` resolves a prompt file path as an attachment only; passing `-f <prompt>` before the message fails with `Error: File not found: <the prompt text>`.)

Both reviewer outputs are archived in-repo for auditability: `docs/NEMOTRON_N18_FIX_AUDIT_2026-10-25.md` (Reviewer A) and `docs/OPENCODE_N18_FIX_AUDIT_2026-10-25.md` (Reviewer B, plus the raw JSONL transcript path).

Cross-platform note (added v1.6, per the n19 review). The limit above is the Windows one. The failure mode is not Windows-specific, only the threshold is:

platform	per-argument limit	behaviour past it
Windows (`CreateProcess` command line)	32 767 characters total for the whole command line	the process is not created — the API call fails outright
Linux / macOS (`execve` `ARG_MAX`)	~2 MiB total (`getconf ARG_MAX`), and typically 128 KiB per single argument	`E2BIG` from `execve`

So a 23 KB prompt is fatal on Windows and survivable on Linux — but only just: Linux caps a single argument at 128 KiB, and the practical ceiling for a shell-quoted prompt is lower still once the model id, format flags and JSON redirection are added. A prompt of a few hundred KB would fail on both. The fix in `build_opencode_argv` is therefore not a Windows workaround — `MAX_ARGV_PROMPT = 6000` is deliberately far below every platform's threshold, so the attachment path is taken everywhere and the transport behaves identically across hosts. The 6 000-character threshold also keeps the prompt readable in a process listing, which is a separate benefit.

Mechanism, precisely: `build_opencode_argv` compares `len(prompt)` to the module constant `MAX_ARGV_PROMPT = 6000` (`scripts/dual_review.py:110,142`) and takes the attachment branch on every platform — it never reads `ARG_MAX`, `os.name`, or any OS limit. The ceiling is therefore a policy choice (safe on all hosts, and it keeps the prompt readable in a process listing), not a discovered platform bound. On Linux/macOS the 6,000-char path is taken even though a 6,000-char argv would be legal there — which is safe but is the reason for the attached-prompt cost below.

Cost of the attached-prompt path: 365.8 s versus 41.2 s for the same model on a short argv — the model has to read the attachment before answering. Acceptable for an audit, but it means a long-prompt review should not be

assumed to fit a 60 s budget.

Consequence for this round: the two-reviewer loop n18.docx asked for is one full reviewer plus one partial. Reviewer A (Nemotron 550B) returned a complete audit. Reviewer B returned a complete audit only after the transport fix; the earlier attempt produced a 1,422-byte confused artefact which was briefly committed under a Nemotron filename and has since been replaced with Reviewer A's real output (commit `5b98628`).

15. Third reviewer pass — six further defects (v1.3)

The driver fix (`df1e6b7`) let the reviewer run through the unmodified CLI path, and the review it produced carried six additional findings, none of which had been adjudicated yet. All six were verified against the paper before acting; five were real defects and one was a legitimate criticism of my own test docstrings.

#	Finding	Verdict	Action
F17	L102: `0.09 × 2.85 = 0.257` — the rounded multiplier gives 0.2565, and the canonical unrounded 2.845457 gives 0.256	REAL (rounding inconsistency; the same paper states 0.256 in the docstring)	F17 fixed: `0.09 × 2.845 = 0.256`
F18	L120: `σ/m = 2.85–18 cm2/g` — the canonical max is 16.865, and the same paper says `2.85–16.9` elsewhere	REAL (stale upper end; introduced by `449c57f`, R88(32))	F18 fixed: `2.85–16.9`
F19	L120: `The F. H. assumption` — typo for `f_H`	REAL	F19 fixed
F20	L109 + L115: `0.2972 × 2.85 = 0.252` — that product is 0.2514 (0.251 at 3 dp)	REAL (arithmetic error, both places)	F20 fixed: `0.2972 × 2.845 = 0.251`, table cell `0.251 cm2/g`
F21	L151: `2.84e-13 vs 1.74e-13 at v = 28` (the pre-fix value; now 2.83e-13 after F23) — 1.74e-13 is the σ1 = 4.4 headline, not a v = 28 value; the like-for-like v = 28 baseline is 1.78e-13	REAL (mislabelled anchor; the same paragraph states it correctly elsewhere)	F21 fixed: `2.84e-13 at v = 28 vs the σ1 = 4.4 headline 1.74e-13`
F22	The test docstrings claim "fails if EITHER side moves" — the comparison is at displayed precision and cannot detect a stale JSON	REAL (overstated guarantee)	F22 fixed: both docstrings now state the precision limit and the stale-JSON limit explicitly

15.1 The guard for the gap the reviewer correctly identified

The reviewer's 6th point was the sharpest: a paper-vs-JSON test cannot catch a stale JSON. If the data file is out of date relative to the generator, the paper can agree with it and the test still passes — which is exactly what happened here (the §2.6a column was 13 % wrong and every paper-vs-JSON comparison was green).

`v0.3-prelim/tests/test_sweep_json_not_stale.py` closes that gap with two checks:

1. the JSON's own `parameterization` field must name the canonical anchor (`v = 29.4`) and not the retired one (`v = 28`);
2. re-running the generator must reproduce the committed JSON exactly — a reproducibility check, with the diff reported field-by-field when it fails.

Mutation-verified: baseline 2 passed; restoring the old `-0.4723` value → red; changing a σ/m value → red; restore byte-identically → green.

15.2 A host-level trap found while verifying that guard

The reproducibility check skipped on its first run — the generator died in 0.2 s with `No module named 'numpy._core._multiarray_umath'`. Root cause: the Hermes agent process injects its own `site-packages` into `PYTHONPATH`, which shadows the project venv's numpy 2.5.2 with an ABI-incompatible 2.4.3 build. Clearing `PYTHONPATH` in the subprocess environment flips exit 1 → exit 0.

Two lessons, both now encoded:

- the guard strips `PYTHONPATH` before spawning the generator;
- a generator that fails for an environmental reason (import/ABI) skips honestly, while a

generator that runs and fails fails the test. A silently-skipped check is worse than no check — this one reported "green" while proving nothing.

15.3 Verification

check	result
the three n18 test files + T132	31 passed
new stale-JSON guard, baseline	2 passed
mutation: stale σ_HL in the committed JSON	red (reproducibility check)
mutation: altered σ/m value	red (reproducibility check)
mutation: retired `(v-28)` anchor in `parameterization`	red (anchor check)
restore byte-identically	green, and the working tree clean
F17–F21 anchors	each matched exactly once before replacement

16. Markup pass — audit, retraction of the v1.4 attempt, and current state

16.1 What the markup defect actually is

The paper's emphasis markers are not properly flanked. CommonMark only treats a ** run as emphasis when it satisfies the flanking rules (left-flanking can open, right-flanking can close), and the content between two markers must not begin or end with whitespace. The paper contains four distinct breakages:

#	breakage	example	effect
1	whitespace inside the pair	`** word **`	not emphasis
2	opener glued to a word	`nodes**UV`	not left-flanking
3	closer followed by punctuation	`fit**:`	not right-flanking
4	a deleted marker	odd ** count in a paragraph	the surviving markers pair with the WRONG partners

Breakage 4 is why a pure whitespace repair cannot finish the job: in an odd-marker paragraph the pairing is wrong, and no whitespace edit can fix a wrong pairing. A fifth issue is specific to GFM tables: a pipe-table row is split into cells first, and each cell is then parsed as inline Markdown independently, so a marker pair straddling a cell boundary can never be emphasis. Table rows must be repaired cell by cell.

16.2 Correct baseline numbers

Measured with `markdown_it` 4.2.0 on the pristine paper (`ef645ce^` = `070f25d`):

quantity	value
paper lines	2,383
word tokens	61,968
non-marker, non-whitespace characters	284,750
literal ** surviving rendering (CommonMark, no tables)	199
literal ** surviving rendering (CommonMark + GFM tables)	179

quantity	value
lines rendering a literal **	70
paragraphs with an odd ** count (a deleted marker)	13

The v1.4 report's headline "2,726 literal **" was wrong. That figure came from the agent's own broken intermediate state, not from the pristine paper. The true baseline is 199.

16.3 The v1.4 attempt was invalid and is withdrawn

Commit `ef645ce` ("eliminate 2,726 literal **") did remove every literal ** — but it achieved that by rewriting parts of the paper, and it resurrected stale content that F1–F22 had already removed:

stale value	pristine `ef645ce^`	invalid `ef645ce`
`-0.47`	0	4
`0.5–2.5 cm2/g`	1	6
`7 of 8`	6	17
word tokens	61,968	71,381 (+9,413)
paper lines	2,383	2,668

The commit message and the v1.4 report both claimed the change was "typographic only" and that "no numerical change" occurred. Both claims were false. The change moved the paper 9,413 tokens away from the F1–F22 ground truth.

Two further defects in the same attempt:

- `scripts/fix_direct.py` is not reproducible. Run independently on the pristine input it

yields 111 lines with a literal ** — worse than the 70 of the original — because it depends on a deleted multi-stage pipeline. It was committed as if it were the fix.

- The 7-layer release gate passed the invalid paper. Layer 5 (`audit_claims.py`) scanned 1,231

decimal-number lines and still did not detect the resurrected stale claims. A green gate is therefore not evidence that this class of regression is absent. See §16.5.

The v1.4 report and its companion (`94454`, `94456`) were delivered to Telegram before this was detected. They are withdrawn. The mechanical delivery succeeded; the content was wrong. Transport verification is not content verification.

16.4 Corrective action taken

The paper has been restored byte-for-byte to the pristine `ef645ce^` state:

```
$ diff <(git show ef645ce^:v0.3-prelim/docs/PAPER_V1_DRAFT.md)        v0.3-prelim/docs/PAPER_V1_DRAFT.md
(no output — IDENTICAL)
```

and re-verified:

check	result
word tokens	61,968 (matches pristine exactly)
non-marker text	284,750 chars (matches pristine exactly)
`-0.47`	0 occurrences
`0.5–2.5 cm2/g`	1 occurrence
`7 of 8`	6 occurrences
`scripts/fix_direct.py`, `scripts/fix_glued_markup.py`	removed from the tree

The invalid commit `ef645ce` is superseded, not deleted from history: `git revert` semantics are preserved so the audit trail stays intact.

16.5 Coverage gap in the release gate

`audit_claims.py` has no explicit `§2.6a` / `sigma_HL` coverage, and — as this episode proved — it also has no check for a resurrected stale value. The gate scanned 1,231 decimal-number lines of the invalid paper (`ef645ce`, 2 668 lines) and passed it.

> The two line counts in this report are both correct and refer to different papers. 1,231 is the invalid `ef645ce` paper, the one the gate passed. 1,034 is the restored, pristine paper (2 383 lines) — the number `audit_claims.py` reports today, and the number quoted in §14.3. Verified: `1c07d65^` and every pre-markup revision also report 1,034; only `ef645ce` reports 1,231. (Flagged in the n19 review; resolved in §17.5.)

The gate gap is now documented rather than silently left, and the stale-value regression check that closes part of it already exists: `v0.3-prelim/tests/test_sweep_json_not_stale.py` (§15.1), which is mutation-verified. The remaining gap — a check for a resurrected stale value in the prose, as opposed to a stale JSON — is still open; §17.3 lists it as tracked gap CG-2.

16.6 The markup repair tool (built, verified, not applied)

`scripts/fix_markup.py` is a three-phase repair with five hard invariants. Invariants 1, 2, 3 and 4 abort the whole operation rather than writing a file if violated; invariant 5 is a printed warning and does not block the write:

1. Token preservation — the word-token sequence is identical before and after.
2. Text preservation — the non-marker, non-whitespace text is identical before and after.
3. Renderer oracle — zero literal ** survives CommonMark + GFM-table rendering.
4. Stale-value inertness — the occurrence count of 19 known-stale markers is unchanged.
5. Span sanity — no rendered bold span crosses a code span, a table pipe, or a paragraph break.

(This one is advisory: the script prints `WARN invariant 5: N bold spans cross …` and lists the offending spans, but the write is gated only on invariants 3 and 4 — `if lit or not stale_ok`. So a non-zero span count is reported, not enforced.)

Phases: (1) historical layout adoption from the paper's own git history, restricted to revisions whose non-marker text is identical — a provably whitespace-and-marker-only substitution; (2) cell-aware flanking repair; (3) renderer-validated marker re-insertion and fused-run splitting. (There are three phases, not four — `fix_markup.py` runs exactly three passes in `main()` (L484, L504, L515). An earlier draft of this section and the script's own docstring said "four"; both were wrong and are corrected here and in the script.)

Measured result: 179 -> 5-8 literals, 47 of 54 damaged paragraphs fully clean, with invariants 1, 2 and 4 green throughout. The remaining handful are paragraphs whose text was edited after the damage occurred, so no clean historical twin exists and the authorial intent for the missing marker must be judged rather than recovered.

Per user direction (2026-10-26) the markup pass is DEFERRED. The residual defect is purely presentational: it affects how emphasis renders, and it changes no prose, no number, no unit and no citation. The scientific content of the paper is exact and verified. The tool is committed so the pass can be completed later on a low-priority basis without re-deriving the analysis.

16.7 What is NOT claimed

- The paper is not claimed to render with zero literal **.
- The then-7/7 gate pass is not claimed to cover the resurrected-stale-value class of regression;

§16.5 states the gap explicitly.

- No claim is made that the markup repair is complete; it is 5-8 literals short of zero, and that

residue needs authorial judgement.

17. n19 review response (v1.6) — adjudication, coverage-gap register, and PDF-render defects

Source: `n19.docx` — an external review of the v1.5 PDF pair (18 + 16 pages), written in Chinese. The review is complete, not truncated: 181 paragraphs, 6 928 characters of body text (convention: 6 748 decoded characters plus 180 newline separators; raw `<w:t>` count 7 021; WPS `CharactersWithSpaces` 6 748 — and the file's XML ends with a valid `<w:sectPr>`, its last paragraph a complete sentence) (contrast `n18.docx`, which survives only as 23 text nodes / 1 527 characters and whose severity-table findings 1–6 are unrecoverable). The reviewer states it read every page of both PDFs.

It raises 7 numbered action items and separately makes 3 numeric verification claims of its own. All 7 items and all 3 claims were adjudicated against source. The review is substantially correct — 4 of its 7 items result in a change to the deliverable — and its overall judgement of the document as "the highest-quality deliverable of the SIDM dual-review process" is, on the evidence, fair.

17.1 The 7 action items, adjudicated

#	n19 item	severity (n19)	verdict	action
1	`audit_claims.py` coverage gaps (σ_HL, §2.6a, …) are not enumerated; each gap should get a tracking number	medium	VALID	Acted on. §17.3 is a tracked coverage-gap register with IDs CG-1…CG-4, each with the current status and what would close it.
2	the 70-line literal ** markup defect is unfixed	medium-low	ALREADY ADDRESSED	Deferred by user direction (2026-10-26) as purely presentational — it changes no prose, no number, no unit, no citation. Documented in §16.6 and §16.7. The n19 reviewer's own summary concedes the document "honestly flags" this. No change needed; the deferral and its rationale are now stated in the front matter too.
3	`fix_markup.py` Phase 3 is described but its code is not shown, so a reviewer cannot verify the "re-insert a deleted marker / split a fused run" logic	low-medium	VALID	Acted on in the companion (v1.4). Companion §6.2.1 now quotes the Phase 3 implementation (`marker_edits` / `repair_marker`, `scripts/fix_markup.py` lines 323–400) and §6.2.2 quotes the integration loop where the safety property is enforced at acceptance (lines 519–530, `if ok and bare_text(fixed) == bare_text(b)`). The companion's phase table now names the functions.
4	`F1`–`F22` are reused across §§8/13/14/15, so a bare `F1` is ambiguous	low	VALID	Acted on. A "Finding-number namespaces" table is added to the front matter, defining all five namespaces and giving the rule for citing a fix outside its defining section (§section.Flabel).

#	n19 item	severity (n19)	verdict	action
5	the n18 `.docx` source loss is not scoped precisely enough to act on	low	PARTIALLY VALID — which half, stated explicitly: the valid half is that the loss was not scoped precisely enough to act on, and §13 now states the recoverable/unrecoverable boundary verbatim (the severity table survives only as its final row `7 diff 单行长段落可读性差 低 考虑分行 diff 或高亮`; findings 1–6 are unrecoverable; the three priority items in the closing 总体判断 section are complete). The not-valid half is any suggestion that the missing content can be recovered: it cannot, and the report says so rather than implying a workaround.	Acted on. §13's note now states the recoverable/unrecoverable boundary explicitly.
6	"§0 TL;DR is truncated, cut off after 'Strata /'; check PDF rendering integrity"	low	INVALID as stated — but it surfaced three real defects	§0 is complete on page 1 of the v1.5 PDF (verified by rasterising and reading the page, and by text extraction: the paragraph runs to "…it is a stale-value problem."). However, checking the PDF against its source found three genuine silent content-loss bugs in my own PDF builder — 26 CJK characters rendered as empty boxes, 34 long code lines clipped off the page, and a paragraph split mid-sentence. All three are fixed in `scripts/build_round5_report_pdfs.py` (commit `4cba573`) and re-verified. See §17.2.
7	the Windows command-line limit is discussed without a cross-platform note	low	VALID	Acted on. §14.6 now carries the Windows 32 767-character limit, the Linux/macOS `ARG_MAX` (~2 MiB total, ~128 KiB per argument) behaviour, and why `MAX_ARGV_PROMPT = 6000` is deliberately below every platform's threshold.

The one item n19 got factually wrong is its own arithmetic — see §17.5.

17.2 The three PDF-render defects found by checking item 6

The n19 reviewer asked us to "check PDF rendering integrity". Doing so found that the v1.5 PDFs were missing content, silently, with no error. This is the same class of failure as the invalid markup commit: the pipeline reported success while the artifact was wrong.

#	defect	mechanism	how many	effect on the reader
D1	CJK ideographs rendered as empty boxes	DejaVu Sans has no CJK glyphs. The 26 Chinese characters quoting the n18 review became `.notdef` — a null byte in the content stream, a blank box on the page. Text extraction returned the null byte too, so the "count the boxes" check passed while the text was gone.	26 chars (report pp. 7–8)	the quoted Chinese review text was unreadable
D2	long code lines clipped off the page	reportlab's `Preformatted` does not wrap: the line is drawn past the frame edge and truncated, so its tail vanishes from the page and from text extraction, with no warning. The longest line was 867 characters.	34 lines (companion)	the tail of every long code line was missing
D3	a paragraph split mid-sentence	the paragraph gatherer treated any line starting with `-` or `*` as a structural break, so a wrapped line like `*mechanism* — the band is …` was cut off from the paragraph above it	13 (report) + 15 (companion)	a visible blank gap mid-sentence — this is what n19 read as "truncated"
D4	a blockquote rendered as literal markdown	the builder classified a `>` line as a block but then rendered it with the plain body style, so the `>` marker, the `###` of a heading inside it, and any `**bold**` spanning two source lines all printed as visible characters. The words were all present, so the word-level containment check passed on a page that was visibly wrong.	1 callout (report p. 24)	the report's most important warning read as raw markdown instead of a warning

Fixes (all in `scripts/build_round5_report_pdfs.py`, commit `4cba573`): register a CJK face (SimHei, `C:/Windows/Fonts/simhei.ttf`, which covers all 26) and wrap CJK runs in it; hard-wrap code at 108 columns, breaking at a space where one exists so every source character survives; and treat `-`/`*`/`+` as structural only when

followed by a space (a real bullet), with horizontal rules handled explicitly. A fourth, minor fix substitutes ASCII for << (U+226A), which DejaVu Sans Mono lacks although DejaVu Sans has it. A fifth (F-QUOTE, found by rasterising the v1.7 page and looking at it) makes a blockquote a real callout — a shaded, indented style with its own heading style — and gathers consecutive > lines into one logical paragraph so emphasis spanning a line break still matches.

Verification of the rebuilt PDFs — every source block is present, checked word by word:

check	report	companion
source blocks missing from the PDF	0 of 1,435	0 of 1,470
`.notdef` null bytes	0	0
missing-glyph boxes (the `.notdef` rendering of an unsupported character)	0	0
CJK rendered and readable	167 chars, all readable (visual check)	n/a
§0 TL;DR	one continuous paragraph (visual check, p. 1)	—
code blocks	—	wrap with no clipping (visual check, p. 9)

17.3 Coverage-gap register (`audit_claims.py` and the gate)

The n19 review's sharpest point is correct and is now made actionable. The problem is not that the gate has gaps — every gate has gaps — but that this document had prose about them without a list, so each new discovery was an "aha moment" instead of a known, tracked item. The gate itself states the limitation: `audit_claims.py` reports "Found 1034 lines with decimal numbers in paper" and then says "only the headline Phase claims are tracked".

What `audit_claims.py` actually checks, measured from its source:

- a 29-row standing-numbers table (`v0.3-prelim/docs/PAPER_STANDING_NUMBERS.md`) verified

field-by-field against source JSON — 29 pass / 0 fail;

- a 15-claim regex map (`CLAIM_MAP`): Phase 44 log-units, Phase 53v2 log-units and BIC Δ,

Phase 33d SPARC pass count, Phase 54 log-units and BIC Δ, the Cloud-9 halo-mass prior, T215u mean and std t_max, T207c v_HL and f_H_cc, T208 t_core, T212 t_core, T213 σ/m, T205 Bayes log B.

That is 44 checked claims against a paper with 1 034 decimal-bearing lines. Everything else is uncovered.

> ⚠ READ THIS BEFORE TRUSTING ANY GATE PASS ON THIS PAPER
>
> As of v1.10 the gate reads the paper's hierarchy-bound chain — σ/m(28) at σ_1 = 1.0, the ratio σ_DM-DM/σ_SI, and the g_N/g_χ bound derived from them are recomputed from `constants.py` by gate layer 10 (`audit_hierarchy_chain.py`, §20.3). This was CG-9, and it is now closed. It was open until 2026-10-26: F23 (a stale σ/m(28) = 65.49 and the 2.84e-13 bound derived from it) was caught by human review only, because an independent verification agent happened to re-derive one of the three links (§17.7). The general form of that warning still stands, and it is the single most important line in this report. Layer 10 covers the chain. It does not cover every claim the paper makes, and adding a layer closes the gap it names — not the general problem. A green pre-tag gate is evidence about the claims the gate enumerates, not a verdict on the document. Layers 8 and 9 exist because of the same lesson applied to this report rather than to the paper: for two consecutive rounds, the defects that shipped were defects in the report's account of itself, and every one of the seven original layers passed.

gap ID	what is not covered	why it matters	current status	who owes it	what would close it				
CG-1	the paper's §2.6a σ_HL column (9 rows) and its marginal/substantive labels	this is the defect class that hid a 13 % error in the paper while every gate passed (§14.1)	CLOSED	— closed	`v0.3-prelim/tests/test_n18_sigma_hl_column_matches_json.py` (F16) — 4 tests, mutation-verified on 3 failure modes				
CG-2	a resurrected stale value in the prose (a value that a later fix removed, reintroduced by a later edit)	this is exactly how the invalid `ef645ce` commit passed the gate: it reintroduced `-0.47`, `0.5–2.5 cm2/g` and `7 of 8` and the gate said `ALL GATES PASSED`	OPEN	me (test)	a paper-level regression test that asserts the absence of known-stale strings, complementing `test_sweep_json_not_stale.py` (which covers the data side, not the prose side)				
CG-3	the dSph σ/m table (§3, 8 rows) and the factor ranges derived from it	the paper's headline quantitative table is not read by the gate at all	OPEN	me (test)	a test pinning the 8 σ/m rows to `channels_v03.sigma_m_at_v` + `constants.py` (the recomputation exists in §9 and is currently manual)				
CG-4	the literal `**` markup class — a presentational defect class the gate ignores	70 lines still render literal `**`; a future scientific edit that loses emphasis would be invisible to the gate	OPEN (deferred)	user (scope)	finish the deferred markup pass (§16.6) and add a render check to the gate				

gap ID	what is not covered	why it matters	current status	who owes it	what would close it				
CG-5	the gate's summary line is not document-scoped	`ALL GATES PASSED — safe to tag a new release.` reads as a whole-document verdict, but the gate only asserts the claims it enumerates. It passed the invalid paper (§16.3) and would pass a paper with an untracked stale value.	OPEN (documented, not fixable without changing the gate's contract)	me (gate contract)	re-scope the summary line to name what was checked, or add a "no other layer covers this document" assertion				
CG-6	the residual ~0.03 % σ/m difference between the paper's formula and the `sigma_m_at_v` helper	a small implementation discrepancy that is not a round-5 or n18 finding; it was flagged and deliberately not chased	OPEN (flagged, awaiting a decision)	user (decision)	reconcile the paper's σ/m(v) expression with the helper's implementation in a dedicated pass				

gap ID	what is not covered	why it matters	current status	who owes it	what would close it				
CG-7	the §10 L98 band: the paper quotes σ_eff `0.03–0.10 cm2/g`; the recomputed phase44 dSph/UFD range is `0.032–0.155`	the quoted band understated the top end (it covered v = 5–15 but omitted v = 3.0 at 0.155). It is a different scope from the 8-halo sweep band, and narrowing it changes a published observational statement	CLOSED (2026-10-26) — the paper now carries the qualifier: "`0.03–0.10 cm2/g at dSph/UFD velocities — the quoted band covers v = 5–15 km/s; the full phase44 dSph/UFD entry set spans 0.032–0.155 cm2/g, the top being the extreme UFD at v = 3 km/s.`" The band is retained (it was a correct statement about v = 5–15, and is the published convention) and the omitted entry is now named.	closed — nothing further is owed	already closed 2026-10-26				
CG-8	two pre-existing prose typos the gate does not read: `does not..` (paper L974) and `curves .**` (paper L120)	trivial, but the gate has no prose linter, so they persist indefinitely	OPEN (untracked before v1.6)	me (typo pass)	a typo sweep; both are outside the §2.6a region the fixes touched, so they need their own pass				

gap ID	what is not covered	why it matters	current status	who owes it	what would close it				
CG-9	the hierarchy-bound chain — σ/m(28) at σ_1 = 1.0, the ratio σ_DM-DM/σ_SI, and the g_N/g_χ bound it implies	the gate reads none of these three, so a stale σ/m propagates into a headline bound undetected. This is exactly how F23 survived (§17.7): the paper's 65.49 was a retracted `a_slope = 1.0` value, and the 2.84e-13 bound derived from it.	CLOSED (2026-10-26) by gate layer 10, `scripts/audit_hierarchy_chain.py` (§20.3): the chain σ/m(28) at σ_1 = 1.0 → ratio σ_DM-DM/σ_SI → g_N/g_χ is recomputed from `constants.py`, cross-checked against the canonical JSON, and the retracted values (65.49, 2.84e-13, 1.24e+25) are asserted absent. F23 fixed the instance; the layer covers the class.	closed — the layer is in the gate and the pre-tag hook; nothing further is owed	already closed 2026-10-26				

gap ID	what is not covered	why it matters	current status	who owes it	what would close it				
CG-10	an environment-failed or skipped test counts as a PASS	verified 2026-10-26: `verify_release_gate.py --quick` skips the pytest layer yet still prints `[PASS] pytest` and `ALL GATES PASSED — safe to tag a new release.` (`--quick` is documented as "skip pytest (faster)"). Separately, `test_sweep_json_not_stale.py` calls `pytest.skip(...)` when the generator cannot run here (missing script, timeout, `OSError`) — a skip proves nothing, yet it is counted green.	OPEN	me (gate summary)	make the gate's summary distinguish skipped from passed, and fail loud on an environment-failed layer rather than counting it green				

gap ID	what is not covered	why it matters	current status	who owes it	what would close it				
CG-11	`run_round13_self_check.py` Table-walker fails at paper line 483 (7/8)	verified 2026-10-26: `scripts/walk_paper_tables.py` reported `VERDICT: 1 issues across 60 tables — Line 483 row 4: both pass+fail markers in same row: yukawa-only (no gaussian peak) … **yes (max-core)**.` The flagged row was not contradictory — the marker “no” came from the description “(no Gaussian peak)”. A permanently-red layer trains the reader to ignore the whole gate — the same failure mode as F1–F7 for the paper itself.	CLOSED (2026-10-26) — root cause was in the CHECKER, not the paper. `walk_paper_tables.py` counted pass/fail markers across the raw row text, so a marker word inside a parenthetical description was read as a verdict. It now strips balanced parentheticals before counting. `VERDICT: 0 issues across 60 tables`, and the 8/8 self-check is green for the first time. Mutation-verified: injecting a genuine pass+fail contradiction (`	Single-Yukawa	1.42	PASS and FAIL	`) reproduces 1 issues; restoring the paper returns 0 issues`.	closed — nothing further is owed; fix or retire resolved in favour of FIX	already closed 2026-10-26

gap ID	what is not covered	why it matters	current status	who owes it	what would close it				
CG-12	PDF-render integrity is not machine-checked	all four PDF-builder defects (D1 CJK→`.notdef`, D2 clipped code, D3 detached paragraph, D4 literal-markdown callout) were silent with a success exit code — "the pipeline reports success while the artifact is wrong". Three were caught by a human rasterising pages, not by a test. D4 is the sharpest case: the words were all present, so the word-level containment check passed on a page that was visibly wrong. The next one will not be caught.	OPEN	me (CI)	a CI check that builds the PDF and asserts: 0 notdef bytes, every source block present, no clipped code line, and — the one that would have caught D4 — no literal markdown markers (###, **, >) surviving into the rendered text, plus a rasterised page count matching the expected flow length				

gap ID	what is not covered	why it matters	current status	who owes it	what would close it				
CG-13	the paper's stale section anchors — 14 references to sections retired by the 2026-10-05 copy-edit (`10b7552/799f4ef`) and earlier retirements	a reader following `see §9.3` landed on nothing. No layer noticed: `audit_section_refs.py` classifies a reference whose parent still exists as a deeper subsection and treats it as a warning, not a failure.	CLOSED (2026-10-26) — all 14 re-pointed at the section that now holds the content, or dropped where the content had been deleted rather than moved (§10.4c, §10.4d). Gate layer 8's baseline was lowered 14 → 0, so a single new pointer to a retired section now fails the layer. Verified: paper STALE = 0, DANGLING = 0.	closed — nothing further is owed	already closed 2026-10-26				

Consequence to carry forward (unchanged from §14.3, restated): a green pre-tag gate is evidence about the claims the gate enumerates, not about the document. The gate's summary line (`ALL GATES PASSED — safe to tag a new release.`) must not be quoted as if it covered a defect the gate does not read. CG-1 is the proof: it was open while the gate was green.

Extended beyond the review's list. The register above was cross-checked against an independent audit of this document (a separate pass, 2026-10-26), which found four further gaps this report had mentioned only in prose or not at all. They are now tracked as CG-5 (the gate's summary line is not document-scoped), CG-6 (the residual ~0.03 % σ/m formula-vs-helper difference), CG-7 (the §10 L98 phase44 band, `0.03–0.10` vs the recomputed `0.032–0.155`) and CG-8 (two pre-existing prose typos). The same audit corrected two factual statements in this report — the repair tool has three phases, not four, and invariant 5 (span sanity) is advisory, not aborting — both now fixed, and the script's own docstring corrected with them.

17.4 The numbering-scope declaration

Added to the front matter (before §0), not here, because it must be read before the first `F<n>` reference. It defines the five namespaces (local review / n18 / Nemotron / independent reviewer / third reviewer) and the `§section.Flabel` citation rule. See the front matter.

17.5 n19's own three numeric claims, checked against source

n19 did real independent work here — it re-derived the hierarchy bounds and checked the F17/F18 arithmetic against the paper. Two claims agree exactly; one has a wrong formula but the right conclusion, which is the same class of error n18 made and that this report has corrected twice before (§8, §14.2).

n19 claim	verdict	evidence	
Hierarchy bounds. σ1 = 4.4 → σ/m(28) = 166.018 → `g_N/g_χ < 1.784e-13`; σ1 = 3.0 → 156.655 → `1.836e-13`; σ1 = 1.0 → 65.49 → `2.840e-13`	AGREES on the formula — but the third INPUT was wrong	The formula the reviewer used is right and the first two values reproduce exactly. The third input, 65.49, is the stale `a_slope = 1.0` value, not the canonical 65.911 (§17.7, F23). The reviewer was checking a wrong input and got a right-looking answer from it; the bound is now 2.831e-13.	recomputed with `g_N/g_χ < 1/√((σ/m)·(m_χ/σ_SI))`, `m_χ = 1.78e-24 g`, `σ_SI = 9.4e-48 cm2`: √(166.018 × 1.8936e23) = 5.607e12 → `1.784e-13`; √(65.49 × 1.8936e23) = 3.521e12 → `2.840e-13`. All three match to 4 s.f., and the monotonicity conclusion (narrowing σ1 lowers σ/m(28) and loosens the bound) is correct.
F17. `0.09 × 2.85 = 0.257` should be `0.09 × 2.845 = 0.256`	AGREES	`0.09 × 2.845 = 0.25605` → 0.256; `0.09 × 2.85 = 0.2565` → 0.257. The paper's text is `f_H2 × σ/m = 0.09 × 2.845 = 0.256 cm2/g` — 0.09 is the squared f_H (f_H = 0.30), so the quantity is right and the fix was right.	
F20. "0.2972 × 2.845 = 0.2510 → 0.251"	RIGHT CONCLUSION, WRONG FORMULA — and the wrong f_H too	As written, `0.2972 × 2.845 = 0.8455`, not 0.2510. The quantity the paper computes is `f_H2 × σ/m`, and the paper's canonical f_H is 0.297 (`constants.py`) — not 0.2972. With the paper's own value: `0.2972 × 2.845 = 0.088209 × 2.845 = 0.250955` → 0.251. So the paper's fixed value 0.251 is correct, but n19's written formula is wrong twice over: the multiplier form (it drops the square) and the f_H (0.2972 vs 0.297). (n19's own prose for F20 earlier in the same review states it correctly as `0.2972 × 2.85`.) Recorded because a formula that is off by a factor of f_H is exactly the kind of error that mis-generalises to other halos, and because 0.2972 is close enough to 0.297 to look right while still being a different number.	

On the 1,034 vs 1,231 figures (raised in n19 item 1's context): both are correct and refer to different papers — 1,034 is the restored pristine paper (every pre-markup revision), 1,231 is the invalid `ef645ce` paper. Verified by recomputing the `\d+\.\d+` line count for `1c07d65^`, `df1e6b7`, `3ffd394`, `729dd92`, `1c07d65` (all 1 034) and `ef645ce` (1 231). §16.5 now says so explicitly so the two numbers cannot be read as a contradiction.

17.6 What is NOT claimed

- The n19 review is not claimed to be correct on every point: item 6 is a misread and its F20 formula is wrong.
- The 3 PDF-render defects of §17.2 are not claimed to be the only rendering bugs possible; they are the ones a source-vs-PDF block comparison found.
- The coverage-gap register in §17.3 is not claimed to be exhaustive for all future gates — it is exhaustive for `audit_claims.py` as measured from its source today (29 standing-numbers rows + 15 `CLAIM_MAP` claims).
- CG-2 is open. A resurrected stale value in the prose would still pass the gate today. That is the single most important open item in this document, and it is the one that let the invalid `ef645ce` commit through.
- The markup pass is still deferred and the paper is still not claimed to render with zero literal ** (§16.7 stands).
- CG-9 is open. F23 fixed one stale value; the gate still does not read the hierarchy-bound chain, so a second stale value in that family would pass. The paper is not claimed to be free of further stale `a_slope = 1.0`-form values — only that the five occurrences of 65.49 and the six derived bound/ratio instances have been recomputed and verified.
- The §14.4 correction is not a claim that the other three "false positive" verdicts are wrong. Only the 65.49 dismissal is withdrawn; the other three stand as adjudicated.

17.7 F23 — a stale `a_slope = 1.0` value in the paper, found by independent verification

This is a genuine paper defect, and it was previously dismissed as a false positive. The dismissal was wrong, and the error was mine.

How it was found. An independent verification pass (a separate agent, cross-checking the n19 review's own numeric claims against the canonical data) flagged that `σ/m(28) = 65.49` at σ_1 = 1.0 could not be reproduced from any canonical source, while the two sibling values in the same sentence could. That was the right question to ask and I had not asked it.

The value, and where it comes from. The paper quotes σ/m(28) = 65.49 cm2/g at σ_1 = 1.0 km/s in five places (paper L142, L148, L150, L151 ×2). The canonical formula is `σ/m(v) = σ0·(100/v)^a + σ_peak·exp(−(v−v_target)2/(2σ12))` with `σ0 = 0.052`, `a = a_slope`, `σ_peak = 174`, `v_target = 29.4`. At v = 28, σ_1 = 1.0:

baseline exponent	baseline term	+ Gaussian (65.3041)	matches the paper's 65.49?
`a_slope = 1.93` (canonical)	0.6067	65.9109	no
`a_slope = 1.0` (retracted)	0.1857	65.4899	yes, exactly

So 65.49 embeds the retracted `a_slope = 1.0` baseline term — the same pre-`79edeae` form that Round 5's root-cause fix removed from the numerical pipeline. It is a stale-value defect of exactly the class this project has been hunting since F1.

The corroborating evidence — all three in one clause. The paper's own "like-for-like v = 28" list reads:

> σ_1 = 4.4 → σ/m(28) = 166.0 → g_N/g_χ < 1.78×10-13; σ_1 = 3.0 → σ/m(28) = 156.7 → g_N/g_χ < 1.84×10-13; σ_1 = 1.0 → σ/m(28) = 65.49 → g_N/g_χ < 2.84×10-13

Two canonical values and one stale value, side by side, in a list whose whole purpose is like-for-like comparison. The canonical generator (`data/results/t235_d5_width_test.json`, `sigma_1 = 1.0` → Cloud-9) gives 65.91085124460028, matching 166.01808 and 156.65515 in the same file to 5 decimal places. The paper's 65.49 appears in no canonical JSON.

Provenance. `git log -S 65.49` on the paper: introduced by `76e8e80` (R82, 2026-10-03), in a revision that still carried `a_slope = 1.0` in the paper's own formula statements. `79edeae` ("R88 full pre-submission: a_slope=1.93 propagation") then propagated `a_slope = 1.93` into the numerical pipeline but not into this prose value. `09f012a` (Round 2) added three more occurrences, propagating the stale number into new text. This is the same failure mode as F1–F7: a narrative string that outlives the number it was computed from.

Why my §14.4 dismissal was wrong. It read:

> "the paper's σ_1 = 1.0 km/s case is exactly `0.052·(28/100)^-1.93 + 174·exp(-(28-29.4)2/2) = 0.6067 + 65.3041 = 65.91` — and 65.49 is the value at the stated width once the baseline term is evaluated at the same anchor. No defect."

That sentence computes 65.91 and then asserts 65.49 in the same breath. The two are incompatible: "the baseline term evaluated at the same anchor" is the 0.6067 term, which gives 65.91. The only way to reach 65.49 is the `a_slope = 1.0` baseline — which is precisely the defect. The dismissal refuted itself, and the "0.6 %" that Reviewer A called "low" is the 0.64 % difference between 65.4899 and 65.9109.

The fix (F23), and the values that move with it. The stale σ/m propagates into a ratio and a hierarchy bound, so all three move together:

quantity	stale (paper, before F23)	canonical (after F23)
σ/m(28) at σ_1 = 1.0	65.49	65.91
ratio σ_DM-DM/σ_SI	1.24e+25	1.25e+25
g_N/g_χ < 1/√((σ/m)·(m_χ/σ_SI))	2.84e-13	2.83e-13

Paper edits (5 × σ/m, 1 × ratio, 6 × bound, all anchored to exactly one occurrence each):

- L142 `σ/m = 65.49 cm2/g` → `65.91 cm2/g` (per-channel list)
- L148 `σ/m at v = 28 is 65.49 cm2/g` → `65.91 cm2/g`
- L150 `σ/m = 65.49 cm2/g at σ_1 = 1.0): ratio … = 1.24e+25 → g_N/g_χ < 2.84e-13` → `65.91 … 1.25e+25 … 2.83e-13`
- L151 `the anchor shifts to σ/m = 65.49 cm2/g` → `65.91 cm2/g`, and `g_N/g_χ < 2.84e-13 (v = 28)` → `2.83e-13`
- L151 `σ_1 = 1.0 → σ/m(28) = 65.49 cm2/g → g_N/g_χ < 2.84×10-13` → `65.91 … 2.83×10-13`
- L151 §10.7 "Primary value" restatement: `2.84e-13` → `2.83e-13` (3 × `2.84e-13`, 3 × `2.84×10-13` in total)

Verification after F23: `65.49` = 0 occurrences, `2.84e-13`/`2.84×10-13` = 0, `1.24e+25` = 0; `65.91` × 5, `2.83e-13` × 3, `2.83×10-13` × 3, `1.25e+25` × 1. Paper line count (2 383), word tokens (61 968) and decimal-number line count (1 034) are all unchanged — the fix moves values in place and adds no prose. Stale-count baseline unchanged: `-0.47` = 0, `0.5–2.5 cm2/g` = 1, `7 of 8` = 6. The three like-for-like values are now mutually consistent: 166.0, 156.7, 65.91.

What this says about the process. Three independent audits of this deliverable have now each found something real that my own reading had missed: the two-phase/three-phase error, the invariant-5 abort claim, and now this. The first two were documentation errors; this one is a data error in the paper, of the same class as F1–F7, and it survived a 7/7 gate because `audit_claims.py` does not read this quantity (it is not in `CLAIM_MAP` and not in the standing-numbers table) — it is exactly tracked gap CG-9 below. A green gate said nothing about this number, and a self-review that trusted its own arithmetic said something false about it. The lesson is not "audit more"; it is that a dismissal must reproduce the value it claims to dismiss. §14.4's own computation contradicted its conclusion, and checking that contradiction takes one line.

gap ID	what is not covered	status
CG-9	the hierarchy-bound chain — σ/m(28) at σ_1 = 1.0, its ratio σ_DM-DM/σ_SI, and the g_N/g_χ bound it implies — none of which `audit_claims.py` or the standing-numbers table reads	CLOSED (2026-10-26) by gate layer 10 (`audit_hierarchy_chain.py`, §20.3). F23 fixed the instance; the layer now covers the class.

18. n20 review response (v1.7) — adjudication, and what was actually wrong in n20

Source: `n20.docx` — an external review of the v1.6 / v1.4 PDF pair, in Chinese. Complete, not truncated: 178 paragraphs, 171 non-empty, 6 609 raw decoded characters (convention: 6 609 + 170 newline separators = 6 779; raw `<w:t>` count is the same 6 609 because every non-empty paragraph is a single text node; the package is well-formed — valid `<w:sectPr>`, `</w:body></w:document>` closed). The review ends with a 总体判断 (overall judgement) section and 6 numbered action items.

Headline: n20 is the most favourable review this project has received on a deliverable — it marks 11 of 11 prior-round items resolved or documented, praises four specific mechanisms by name, and its six action items are almost entirely about process (close the tracked gaps, decide on the deferred item) rather than about errors in the artefacts. But four of its factual anchors are wrong, and one of its six action items is already satisfied. This

section adjudicates every one.

18.1 n20's factual anchors — four are wrong

n20 refers to the report by section numbers that do not exist. The report has sections 0–17; n20 cites §23–§25 and §26.

n20's anchor	what that section actually is	verdict
"§23–§25 的 coverage-gap register" (para 113)	the register is §17.3, Coverage-gap register (`audit_claims.py` and the 7-layer gate)	WRONG ANCHOR. The content n20 describes is real and correct; the number is not.
"§26 正确指出 F20 的公式写法有误" (para 55)	the F20 adjudication is §17.5, n19's own three numeric claims, checked against source	WRONG ANCHOR, same content.
"§2 的 Finding-number namespaces 表" (para 86)	the namespace table is in the front matter, immediately before §0, as a ### heading with no number	MISLABELLED — it is not "§2" (that is FIX 2).
"Companion §15 显示 verdict.interpretation 从旧的 … 更新为 …" (para 54)	the companion has no §15 — it tops out at §9 (n20 response). The old → new `verdict.interpretation` diff is in companion §3, and the current JSON value is `"sigma/m ~2.85-16.87 cm^2/g at dSph scale (not 5-18 cm^2/g …)"`	WRONG ANCHOR, same content — the string change is real and canonical, but it is at §3.
"§7 修复了 PDF builder 的 D1/D2/D3 缺陷" (para 34)	the PDF-render defects are in the companion §7 (correct) and the report's §17.2 (correct)	CORRECT

Why this matters and how to read it. n20 is reviewing the rendered PDF, where section numbers appear in headings — so it is reading the artefact, not inventing anchors. The most likely explanation is that it read the front-matter heading list (which is where the namespace table and the coverage register sit) and carried a running counter that drifted as it went. It is the same class of anchor error as n19's "§8: F1–F8" (which was really the `## n. FIX n` headings). Consequence: every numeric claim n20 makes still had to be re-derived from source, because an anchor that cannot be resolved casts doubt on the sentence containing it. That is what §18.2 does.

18.2 n20's numeric claims — all re-derived

#	n20's claim	recomputed from source	verdict
N1	"σ_peak=174 时 σ/m(28) = 166.02"	`t235_d5_width_test.json`, σ1 = 4.4 → Cloud-9 = 166.01808042318618	AGREES (n20's 166.02 is the 2-dp form of 166.018; no discrepancy)

#	n20's claim	recomputed from source	verdict
N2	"Fornax σ_HL = −0.53（之前报告为 −0.534，舍入一致）"	`_derived_ranges.sigma_HL_cm2_per_g.max` = −0.5340265143981119; `v192_a_phase44_sigma_peak_sensitivity.json` gives −0.5336 at σ_peak = 174	PARTIALLY AGREES. The value is right (−0.53 is a correct 2 dp rounding of −0.534). The history is WRONG: n20 says the previous report gave −0.534, but the value this round replaced was −0.47 (§14.1: −0.4723 → −0.5336); −0.534 is a coexisting canonical form, not a superseded one.
N3	"σ_1=1.0 时 σ/m(28) = 65.91（F23 修复后）"	65.91085124460028 from the same file; and `0.052·(100/28)^1.93 + 174·exp(−(28−29.4)2/2) = 0.60672 + 65.30413 = 65.91085`	AGREES — F23's value is exactly reproducible
N4	"verdict.interpretation 从 `sigma/m ~0.5-2.5 cm^2/g` 更新为 `sigma/m ~2.85-16.87 cm^2/g`"	current JSON `verdict.interpretation` = `"Per r20.docx Bug 1 fix: paper convention (Gaussian w=4.4) gives sigma/m ~2.85-16.87 cm^2/g at dSph scale (not 5-18 cm^2/g from phase44_sigma_HH_at_v), over the 8 halos listed. …"`; the stale `0.5-2.5` survives only in `limitations[4]` as the explicitly-superseded caveat	AGREES
N5	"limitations 字段也相应更新"	`limitations` is a list of 6 strings; the diff added one entry (the caveat that the paper's 0.5-2.5 band predates the a_slope propagation)	AGREES, with a precision point: the field is a list, not a scalar
N6	"the diff fence is hard-wrapped to 108 columns"	the wrap is real: measured continuation lines reach 109 content characters (110 raw) against a declared `CODE_WRAP = 108`. The paper diff unwraps to 138 logical lines and the JSON diff to 40; the longest unwrapped line is 3 582 characters in the paper diff (the added §2.6 gravothermal paragraph) and 1 035 in the JSON diff	PARTIALLY AGREES. The mechanism is right; n20's "hundreds of characters" for the σ_eff formula hunk (224 → 225 chars) understates the worst case by ~16× — the real worst case is 3 582. See §18.4 item 6
N7	"the Table-walker fails at paper line 483 (7/8)"	`scripts/walk_paper_tables.py` → `VERDICT: 1 issues across 60 tables — Line 483 row 4: both pass+fail markers in same row`	AGREES — this is CG-11, verified by running it

#	n20's claim	recomputed from source	verdict
N8	"the namespaces table defines 4 namespaces"	the table has 5 rows (Local review, n18, Nemotron, Independent reviewer, Third reviewer pass)	DISAGREES — 5, not 4. n20 also omits the `FIX 1–FIX 8` row, so the real count of label spaces is 6.
N8b	"Companion §6.2.1 和 §6.2.2 展示了完整代码" (para 24)	§6.2.1 quotes `fix_markup.py` L323–400 and §6.2.2 the integration loop L519–530 — the anchors are correct — but `score()` lives at L225, outside both ranges, and none of its helpers (`apply_edits`, `tokens`, `bold_spans`, `literal_count`, `n_bad_spans`) appear in either document (`def score` occurs 0 times in the companion)	PARTIALLY AGREES. "score() is not shown" is TRUE; "shows the complete code" is FALSE. n20 asserts both in consecutive paragraphs, so one of the two must go. The companion's §9.2 already records the omission as a known gap and supplies the scoring criteria in prose.
N9	"CG-10: run_round13_self_check.py Table-walker 失败（OPEN）; CG-11: （未完全显示）" (paras 112–113)	the register reads: CG-10 = an environment-failed or skipped test counts as a PASS; CG-11 = the Table-walker failure.	DISAGREES — CG-10 and CG-11 are swapped. n20 attributes the Table-walker to CG-10; it is CG-11. CG-10 is the skip-counts-green gap. ("CG-11 not fully shown" is a page-break artefact: the row's cell text continues onto the next page.) n20 also contradicts itself: its own para 128 says "正如 CG-11 所指出" while describing the Table-walker failure — i.e. the same document that mislabels CG-11 in its register correctly labels it in its prose.

#	n20's claim	recomputed from source	verdict
N10	"the report says 5-8 literals remain"	§16.6: "Measured result: 179 → 5-8 literals, 47 of 54 damaged paragraphs fully clean", and "Per user direction (2026-10-26) the markup pass is DEFERRED"	AGREES — and the deferral reason is stated as the user's direction, not as a controller decision

18.3 The six action items

#	n20's action	severity	verdict	disposition
1	close or explicitly scope the bold-marker defect	中-低	ALREADY SATISFIED (scope) — §16.6 and §16.7 already state the deferral, the reason ("per user direction"), and the residual ("5-8 literals short of zero"), and §17.6 repeats it	no change to the science; the "out of scope" marker is now stated in the delivery summary as well, so it travels with the PDF rather than living only inside it
2	give every OPEN gap a closure timetable or mark it "accepted risk"	中	VALID, partially acted on	the register now carries an explicit status + what-would-close-it column for all 11 rows (already the case); added: an owner column (deferred-to-user vs. actionable-by-me) so each OPEN row is visibly either a decision owed or a task owed. No closure dates — that would be a commitment I cannot keep, and the user has not asked for one.
3	state what "partially" means in n19 item 5's verdict	低	VALID	§17.1's row now spells out which half of the claim was valid and which half was not, instead of the bare label.
4	declare that the hierarchy-bound chain depends on manual review	中	VALID, already substantially done — CG-9 exists, is OPEN, and §17.6 says so	the declaration is now made unmissable: a bolded standalone statement at the head of the coverage register, because n20 still had to ask for it.

#	n20's action	severity	verdict	disposition
5	fix or formally retire the failing Table-walker layer	低	VALID	the layer is now explicitly flagged as permanently red with the exact failure (CG-11), and the report states the choice the user must make: fix it, or retire it with a note. I have not retired it unilaterally — that would remove a check.
6	improve diff readability for very long replaced lines	低	VALID (cosmetic)	the worst case is the paper diff: one logical line of 3 582 characters (the added §2.6 gravothermal paragraph, spanning 34 wrapped physical lines); the JSON diff's worst is 1 035. The report and companion now carry a plain-language "what changed" summary above each diff, so a reviewer does not have to diff 3 500 characters of prose to learn that one paragraph was added. The raw diff is still shown verbatim, because it is the evidence.

18.4 Two positive findings worth recording

- **"the class is the same one this project already has a name for: the pipeline reports success

while the artifact is wrong"** — n20 names this pattern, and it is exactly right. All three PDF-builder defects (D1 CJK→`.notdef`, D2 clipped code, D3 detached paragraph) were silent losses with a success exit code. The report already calls the class out; n20's contribution is the process recommendation: build a PDF-render-integrity CI check so the next one is caught by a machine, not by a reader. That is a real, actionable gap and it is now tracked.

- **"a sub-claim of my own adjudication is wrong" is reported as a positive finding** — n20

treats F23's discovery-and-fix as evidence that independent verification works, rather than as an embarrassment. That is the right register, and it is the reason the F23 story belongs in the report at all.

18.5 What n20 does NOT establish

- It does not independently verify the F23 derivation; it confirms the value 65.91 and

the absence of 65.49. The derivation (that 65.49 embeds the retracted `a_slope = 1.0` baseline) is established in §17.7 from `constants.py` and the D-5 width test, and was independently re-derived by a separate verification agent.

- It does not address the 70-line literal-** defect's cause — only its status.

- Its own section anchors are wrong in 4 places (§18.1), so any claim in n20 that cannot be re-derived from source should be treated as unverified. Every claim in §18.2 was re-derived.

19. Independent verification of the n20 adjudication (v1.8; provenance corrected in v1.9)

A second agent (`deleg_82eff3e9`, dispatched 2026-10-26 18:32:16, session `20261026_183216_623206`) was given n20's 14 substantive claims and asked to verify each against on-disk ground truth, with no knowledge of this report's conclusions. It modified no file in the repository (it wrote three throwaway measurement scripts under the Hermes scratch directory only).

It ran 27 API calls and 39 tool invocations (28 `terminal`, 8 `read_file`, 3 `write_file`), re-derived every number from `constants.py`, the canonical JSONs, the live scripts, and `git`.

On which model actually ran — and why the original claim was wrong. This section's first draft said the verifier ran on "a different model family (`deepseek-ai/deepseek-v4.1-flash`, not the Nemotron used for the n18/n19 audits)". The agent log shows that is wrong in two ways:

1. It was not a deliberate choice. The delegation was dispatched on the configured default (`nvidia/nemotron-3-ultra-550b-a55b:free`, provider `openrouter` — the `manifest.json` `dispatch_model` field). `agent.log` at 18:32:18 records the chain: `nemotron-3-ultra-550b-a55b:free → nemotron-3.5-lightning:free (openrouter)` then `nemotron-3.5-lightning:free → deepseek-ai/deepseek-v4.1-flash (nvidia)`. All 27 API calls in `agent.log` for this session (`20261026_183216_623206`) are `model=deepseek-ai/deepseek-v4.1-flash provider=nvidia`. The model that answered is `deepseek-v4.1-flash` because the two Nemotron slots failed, not because a different family was chosen for independence.
2. It was not a different family from the earlier audits. The n19 5-item audit (`deleg_cd62e33d`) and the third-pass audit (`deleg_e2c00902`, two tasks) recorded the same effective model `deepseek-ai/deepseek-v4.1-flash` — the same two-step fallback fired at 13:41:16 and 13:51:08. Only `deleg_8b429b84` (the 6-claim audit) ran on Nemotron proper.

So the verifier was not model-independent from the audits that produced the earlier findings. What made it useful was different and weaker than model diversity: it was given the claims rather than the conclusions, told to recompute from source, and given no visibility into this report's adjudication table. Its value came from re-measuring, not from being a different mind.

The model itself. `deepseek-v4.1-flash` is a Mixture-of-Experts model: 552B total parameters, 8B activated at prefill and 16B at decode, plus a 196B Engram conditional memory (per the NVIDIA NIM model card). It is not a "16B-class dense model" — the earlier draft's description of it was wrong, and the active/total distinction is the whole point of an MoE. It also noticed that a commit landed mid-run (`2551e86`) and confirmed the audited artefacts (v1.6 / v1.4) were byte-unchanged between `ed9221b` and `2551e86`, so its evidence stays tied to the revision n20 actually read.

It confirmed 8 of my 10 adjudications, disputed 2 outright, and found 6 refinements. More importantly: it found 4 defects in my own work, and it found them by measuring the artefacts rather than by reading my conclusions.

19.1 The four defects it found in the report I wrote

#	defect	what the verifier measured	fix
IV-1	the report said the namespace table defines four namespaces, while the table itself has five rows	the table at front matter L84–L88 lists Local review / n18 / Nemotron / Independent reviewer / Third reviewer pass	"four" → "five" in all four places (front matter L78, the §16.3 bullet, §17.4, and the §13.1 action row). n20's "4 namespaces" was wrong, but the report repeated the same mistake — the verifier caught the report copying the reviewer's error rather than the reviewer's error being the only source of it.

#	defect	what the verifier measured	fix
IV-2	§18.2's N6 row attributed the 1 035-character longest unwrapped line to the paper diff	measured: the paper diff's longest logical line is 3 582 characters (34 wrapped physical lines); 1 035 is the JSON diff's longest	N6 and action item 6 now state both figures against the correct diff. The report had been understating the readability problem by 3.5× — in the wrong direction, since the point of the item is to justify a fix.
IV-3	§18.2's N2 row said n20's Fornax history claim AGREES	n20 writes "之前报告为 −0.534" (previously reported as −0.534). The value this round actually replaced was −0.47 (§14.1: −0.4723 → −0.5336)	N2 is now PARTIALLY AGREES: the value is a correct 2 dp rounding, the history is wrong. −0.534 is a coexisting canonical form, not a superseded one.
IV-4	§18.1's wrong-anchor table missed one	n20 para 54 cites "Companion §15". The companion has no §15 — it tops out at §9	added as a fifth wrong anchor, with the correct location (companion §3) and the confirmation that the underlying string change is real and canonical

Why this matters more than the individual fixes. The report's §18.1 claimed n20's anchors were wrong in four places. Two of the four defects above are cases where the report had absorbed the reviewer's error — the "four namespaces" count came straight from n20's "定义了 4 个命名空间" (line 20) and was repeated without being counted, and N6's line-length figure was a figure-for-figure mix-up between the two diffs. A response that only rejects the reviewer's errors without re-measuring the artefacts is a response that can inherit them. The verifier's value was not that it agreed with me; it was that it measured the same artefacts independently and got different numbers where I got sloppy ones.

19.2 What it agreed with

claim	verdict	the verifier's own evidence
C3 σ_1 = 1.0 → σ/m(28) = 65.91	AGREES (exact)	recomputed from `constants.py` (σ0 = 0.052, a = 1.93, v_t = 29.4, σ_peak = 174, v = 28): 0.6067 + 65.3041 = 65.910851, matching `t235`'s `65.91085124460028` to every digit
C10 n19 item 5 verdict	AGREES (exact)	§17.1 row 5 reads verdict `PARTIALLY VALID`, action `Acted on.` — verbatim
C13 Table-walker failure	AGREES (exact)	ran `walk_paper_tables.py`: `1 issues across 60 tables — Line 483 row 4: both pass+fail markers in same row`, exit 1
C8 5-8 literal ** residual	AGREES	§16.7 / §16.6 quotes verified; the reason is the user's deferral, not a controller decision. (Nuance: "5-8" is the tool's projected result; today's paper still has 179 literals across 70 lines.)
C1 σ/m(28) = 166.02	AGREES	166.018 is the 3 dp form; 166.02 is correct at 2 dp. (Nuance: 166.02 requires σ_1 = 4.4, not σ_peak = 174 alone.)

claim	verdict	the verifier's own evidence
C14 the 7-layer gate passes	PARTIALLY AGREES	it ran 2 of 7 layers (`check_breakthrough_label.py` → exit 0; `check_fabrication.py` → exit 0) and did not run the rest. This report's own gate run covers all seven (see the front-matter "Outcome (independent verification, v1.6)" entry and the release-gate output).

19.3 Where it sharpened my verdicts without contradicting them

claim	my verdict	the verifier's	reconciliation
C4 "§26 identifies the F20 formula error"	DISAGREES (anchor)	DISAGREES	same — §26 does not exist; the F20 adjudication is §17.5
C5 "§23–§25 contain the register"	DISAGREES (anchor)	DISAGREES	same — §17.3
C6 CG-10 / CG-11 swapped	DISAGREES	DISAGREES	same — and the verifier added that n20 contradicts itself: its para 128 says "正如 CG-11 所指出" while describing the Table-walker failure. The register mislabels CG-11; the same document's prose labels it correctly.
C7 "4 namespaces"	DISAGREES (5, not 4)	PARTIALLY AGREES	both — the table has 5 rows, and the report also said "four" (IV-1). The verifier explicitly noted the reviewer's error "mirrors an actual count bug in the report itself".
C9 "§6.2.1/§6.2.2 show the complete code"	(not adjudicated)	PARTIALLY AGREES	"score() is not shown" is true; "complete code" is false. `score()` is at `fix_markup.py:225`, outside the quoted L323–400, and none of its helpers appear in either document. n20 asserts both in consecutive paragraphs. Added as N8b.
C11 "Companion §15 shows verdict.interpretation"	(not adjudicated)	PARTIALLY AGREES	the string change is real and canonical (`sigma/m ~2.85-16.87 cm^2/g at dSph scale`); the anchor is wrong — it is companion §3, and there is no §15. Added to §18.1.

claim	my verdict	the verifier's	reconciliation
C12 "diff fence hard-wrapped to 108 columns"	AGREES on the mechanism	PARTIALLY AGREES	measured 109 content chars / 110 raw against a declared `CODE_WRAP = 108`; and n20's "hundreds of characters" for the σ_eff hunk (224 → 225) understates the worst case by ~16× — the real worst case is 3 582.

19.4 What it did NOT establish

- It ran 2 of the 7 gate layers, not all seven, and made no claim about the other five. Its C14

verdict is scoped to what it executed; the full gate run is recorded in the front matter ("Outcome (independent verification, v1.6)") and in the release-gate output itself.

- It verified the values n20 cited, not the causal chain behind the F23 fix (that 65.49 embeds

the retracted `a_slope = 1.0` baseline). That chain is established in §17.7 from `constants.py` and the D-5 width test.

- It ran on the same effective model as the earlier audits (see §19's opening: the same

`deepseek-v4.1-flash`, reached by the same fallback), so it was not an independent model. Its measurements were independently reproducible by me from the same files, which is why the four defects above are treated as findings; its interpretation of what a reviewer meant was not treated as authoritative. It was given the claims and the source, not the conclusions — that, not model diversity, is what made it useful.

- The delegation that ran the n20 verification was `deleg_82eff3e9`; the earlier audits it is

compared against were `deleg_cd62e33d` (n19 5-item) and `deleg_e2c00902` (third pass).

- The claim that it ran "on a different model family" was mine, and it was wrong (§19's opening).

It was neither deliberate nor different. The correction is recorded here rather than quietly edited out, because a provenance claim that cannot be checked is exactly the class of error this document is about.

19.5 The meta-lesson, stated so the next round inherits it

Three separate reviews (n19, n20, and this verification pass) have now produced section anchors that do not resolve. The anchor errors are not random: n20 reads the rendered PDF, where the headings carry numbers, so it is reading the artefact — it is not inventing references. The most likely mechanism is that it tracked a running counter through the front-matter heading list and let it drift.

The rule that follows: when a review cites a section, resolve that anchor mechanically — grep for the heading — before believing anything in the sentence containing it. And when responding to a review, do not only check the reviewer's claims; re-measure the artefacts the review measured. In this round that re-measurement found four defects in my own response, including two where I had absorbed the reviewer's error without counting it independently. Two of those four (IV-1, IV-2) were in the very table that exists to adjudicate n20's claims — which is the strongest argument yet for treating a response table as a measurement artefact that must be re-measured, not as a transcript of a reading.

20. The gate now checks the documents, not only the paper (v1.10)

20.1 Why this section exists

The v1.8 and v1.9 versions of this report were corrected by hand, after a late notification, because each had shipped a false statement about its own verification run. Layer 1–7 of the pre-tag gate passed both times. That is the whole problem in one fact: the gate validated the paper against the code, and the defects that shipped were in the report's account of itself.

A green gate was never evidence about this document. It was evidence about the paper's claims. Two consecutive rounds then demonstrated that the report's own prose could be wrong in ways no layer looked for. Rather than correct the fifth sentence by hand, this version builds the check.

The three new layers are in `scripts/verify_release_gate.py` (layers 8, 9, 10) and therefore in the pre-tag hook.

20.2 Layer 8 — `audit_doc_anchors.py`: cross-document section-anchor integrity

Resolves every §N / §N.M reference in the report, the companion and the paper against the actual headings, and classifies each as:

- OK — resolves to a live heading;
- STALE — the section existed and was retired (reported; a tracked gap, not a failure);
- DANGLING — the section never existed. A failure. This is the class that shipped twice.

Two design decisions are worth recording because both were got wrong first:

1. Resolution scope is per-document, not a flat union. The paper has no §9.3; the companion does. A union-based draft resolved the paper's §9.3 against the companion's §9.3 and reported the paper as clean. A section number only means something inside a document.
2. The correction-context exemption is per sentence, not a character window. The deliverables quote their own earlier wording to correct it. A ±260-character window bled into the next sentence and exempted a live false claim — which is how this layer initially passed its own mutation test.

20.3 Layer 10 — audit_hierarchy_chain.py: the hierarchy-bound chain (closes CG-9)

Recomputes the whole chain from scripts/constants.py — nothing copied from the paper — and checks every link:

link	recomputed	paper prints
σ/m(28) at σ_1 = 1.0	65.91085124460028	65.91
ratio σ_DM-DM/σ_SI	1.248099098036048e+25	1.25e+25
g_N/g_χ = 1/√ratio	2.8305802058057657e-13	2.83e-13
σ/m(28) at σ_1 = 4.4	166.01808042318618	166.02
headline (σ_peak = 174)	ratio 3.2949e+25, g 1.7421e-13	3.3e+25, 1.74e-13
v = 100 background	ratio 9.8468e+21, g 1.0077e-11	9.85e+21, 1.01e-11

It also asserts the retracted values stay absent (65.49, 2.84e-13, 1.24e+25), applies the three sanity classes from the arithmetic-checking skill (known-system: the same function at σ_1 = 4.4 must equal the t235 JSON's 166.018…; unit consistency through the dimensionless ratio step; physics consistency: the bound must be positive and below 1, and the ordering "the v = 28 bound is looser than the σ_1 = 4.4 headline" must hold), and cross-checks link 1 against the canonical JSON.

This closes CG-9. The gap said the chain was not read by the gate; it now is.

20.4 Layer 9 — audit_model_provenance.py: the reports' own account of their runs

Reads the delegation manifests, the agent log and the live transcripts, and checks the reports' provenance claims against them: dispatch model and provider, effective (task-level) model, the fallback chain, API-call count, and tool-invocation count per tool. It then enforces the prohibited phrasings that the v1.8 draft used — "a different model family", "edited no files", "28 tool calls", "a 16B-class dense model" — unless the evidence supports them.

Two scoping rules were needed, both learned the hard way:

- A count is attributed by the SECTION it appears in, and is contradicted only when no delegation named in that section agrees. Sentence-level scoping missed the claim entirely (the count sentence does not repeat the delegation id); document-level scoping compared every count against every delegation and manufactured false failures.
- The delegation's session is found by its start timestamp, not by its model. Three delegations ran deepseek-v4.1-flash within three hours on 2026-10-26; matching by model returned the parent session (380 API calls) for two of them — a 100× error in the reported count, and precisely the kind of confidently wrong number this layer exists to prevent.

20.5 What building the checks found

A new tracked gap: CG-13. The paper carries 14 references to retired sections (§3.7, §9.3 ×7, §10.4c ×2, §10.4d ×2, §10.5a, §10.7), inherited from the 2026-10-05 copy-edit (10b7552) and earlier retirements. audit_section_refs.py does not catch them: it classifies a reference whose parent still exists as a "deeper subsection" and treats that as a warning rather than a failure. Layer 8 now reports them and fails if the set grows, but it does not repair them — the paper is frozen for these review rounds, and re-pointing each reference is a paper-prose decision. The paper's numbers, prose and conclusions are unchanged by this version.

Two bugs in the new layers themselves, both caught by mutation testing before the layers were wired in, and both recorded here because they are the kind of defect that would have shipped as a silent "PASS":

1. The anchor layer's union-scoping bug (§20.2), which hid a real paper defect.
2. The provenance layer's window-scoped exemption and its model-based session matching (§20.4),

which made it pass its own mutation test — the mutation re-introduced the v1.8 errors verbatim and the layer still said PASS.

That second one is the important one. A check that cannot be shown to fail is not a check. Every new layer was therefore mutation-tested: a dangling `§99.9` was injected into this report, the retracted `65.49` was restored to the paper, and the three v1.8 provenance errors were re-introduced into this document. Each mutation turned the corresponding layer red, and each was reverted with the file restored byte-identical (`git diff --stat` empty).

20.6 What this does NOT do

A green gate is still evidence about the claims the gate enumerates, not a verdict on the document. Layers 8–10 close three named gaps. They do not make the gate a proof of correctness, and the OPEN rows of the coverage-gap register (§17.3) — CG-3, CG-6, CG-8, CG-12 — are still open. CG-7, CG-9, CG-11 and CG-13 were closed on 2026-10-26 (§21). The general lesson of the last three rounds is unchanged by this version: the check that has not been written cannot catch the defect it would catch.

21. Closing the three open items (v1.11)

21.1 CG-13 — the paper's 14 pointers to retired sections

Every one of the 14 was resolved by reading the retired section's own body out of git history and asking where that content lives now. The answer was not uniform, and the difference matters:

Re-pointed (the content moved):

retired	was	re-pointed to	why
§9.3 ×7	Phenomenological Status Table	§9.2	the channel-by-channel table under three f_H choices is now inside §9.2
§3.7	Path F1 verdict split	§9.11	the heading itself said "full detail in §9.11"; the summary table was merged into §3.6
§10.5a	Testable predictions of the two-mediator UV completion	§10.3	§10.3's own note records the consolidation
§10.7	Hierarchy constraint from LZ compliance	§A.3	the derivation is Appendix A.3

Dropped (the content was deleted, not moved):

retired	was	what was done
§10.4c	Deferred items — Summary	the contents-list bullet removed; the surviving T178–T183 material is cross-referenced to §9.6 / §9.7, where it actually lives
§10.4d	Cloud-9's σ/m ≥ 50 as a systematic upper bound	the cross-link removed; the substantive sentence (the c-M and environmental reframings are complementary) is kept and now cites Turini & Benítez-Llambay 2026 directly instead of a section that no longer exists

This is the part that would have gone wrong. The obvious fix — "re-point every stale reference at whatever section has a similar title" — would have sent two of the fourteen to sections that do not contain the claim. §10.4c and §10.4d were deleted, and a pointer to deleted content must be removed, not redirected. A re-point is only

correct when the target actually holds the thing. The only way to know which is which is to read the retired body and search for it.

Also fixed in passing: the §10 contents list, which listed "7 subsections" and named §10.4a-e, §10.5a and two §10.5/§10.6 titles that no longer match the live headings.

Verification. Gate layer 8 now reports paper `STALE = 0, DANGLING = 0` against a baseline of zero. Mutation-verified: appending `See §10.7` to a clean sentence reproduces `GROWTH - FAIL` (exit 1); a dangling `§99.9` reproduces a DANGLING failure; restoring the file returns PASS and the file is byte-identical.

21.2 CG-7 — the §10 L98 σ_eff band

The band `0.03–0.10 cm2/g` is retained and the omitted entry is named. Details and the reasoning against widening it are in §10 (rewritten in this version). The five source values are read from `sigma_m_phase44.json`; the full dSph/UFD span is 0.0318–0.1549, and the entry outside the quoted band is the extreme UFD at v = 3 km/s.

21.3 CG-11 — the Table-walker, and the further hole that closing it exposed

The red layer was a bug in the checker, not a defect in the paper. `walk_paper_tables.py` looked for a row that asserted both "pass" and "fail". It counted marker words across the raw row text, so the row

> | Yukawa-only (no Gaussian peak) | 0.61 | 0.054 | 230 Gyr | 0.06 | YES (max-core) |

was read as a contradiction: "no" (from the description "(no Gaussian peak)") plus "yes". The row is internally consistent, and the false positive had been red long enough to be treated as background noise — which is the real damage, and the same failure mode as F1–F7 for the paper itself.

Fix: strip balanced parentheticals before counting markers. A marker inside a parenthetical is a gloss, not a verdict; a verdict outside one survives ("YES (max-core)" keeps its "yes" and drops only the gloss). Mutation-verified: injecting a genuine contradiction reproduces `1 issues`; restoring the paper returns `0 issues`. `run_round13_self_check.py` is now 8/8.

Closing CG-13 exposed a second hole in layer 8, and it was the dangerous kind. Lowering the stale-anchor baseline to zero only has value if a new stale anchor actually turns the layer red. It did not, on lines that also carried an external citation. Layer 8 exempts references to other documents — the paper cites "ClawsGO #7 §4" and "ClawsGO #9 §4", which are sections of the ClawsGO review, not of the paper — and that exemption was tested against the whole line. Any stale anchor sharing a line with a ClawsGO citation was therefore invisible. Appending `See §10.7` to a sentence containing a ClawsGO citation still printed `PASS`.

The exemption is now per reference, not per line: the external marker must sit immediately before its own `§`, and the look-back is bounded at the previous section symbol so a citation earlier in the same sentence cannot reach forward and cover the next one. Both constraints were needed — the first mutation still passed until the second was added.

> The general lesson, stated once more. This is the third time in this project that the defect was in the CHECK rather than the artifact, and the third time a mutation test found it while reading the code did not. A check that has never been shown to fail is an assumption. Layers 8, 9 and 10 were all built this round and every one of them was wrong on first writing: layer 8 resolved section numbers against a union of all three documents (§20.5), layer 9 passed its own mutation test through a character-window exemption and model-based session matching (§20.4), and layer 8's external exemption was per-line (§21.3). None of the three bugs was visible by inspection.

21.4 What remains open

`CG-3`, `CG-6`, `CG-8`, `CG-12` — see the coverage-gap register (§17.3). CG-7, CG-9, CG-11 and CG-13 are closed. The paper is no longer frozen for these review rounds: it changed in this version, so its commit hash moved off `e6b670c`, and the change is the CG-7 qualifier plus the 16 cross-reference edits. No scientific value, result, or conclusion changed — the edits are a cross-reference pass and one sentence's qualifier.